

Отделение информационных технологий и вычислительной техники

КУРСОВАЯ РАБОТА

по МДК 02.01 Технология разработки программного обеспечения
специальности 09.02.07 Информационные системы и программирование

ПРОЕКТИРОВАНИЕ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ БАЗЫ ДАННЫХ ДЛЯ АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ АРЕНДЫ НЕДВИЖИМОСТИ В РАСПРЕДЕЛЕННОЙ ГОРОДСКОЙ СЕТИ

Исполнитель:
студент группы № ИП-32
_____ / Савоськин М.И

Руководитель:
_____ / Прохорова И.Н.

Курсовая работа
защита с оценкой

Новосибирск, 2026

Оглавление

ВВЕДЕНИЕ	3
ГЛАВА 1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ	5
1.1 Постановка задачи.....	5
1.2 Обзор инструментальных средств.....	6
1.2.1 Выбор системы управления базами данных	7
1.2.2 Выбор сред разработки	8
1.2.3 Выбор дополнительных инструментов	8
1.2.4 Выбор архитектурного подхода	10
1.3 Анализ предметной области и определение возможностей системы	11
1.3.1 Анализ конкурентов.....	11
1.3.2 Уникальность разрабатываемого продукта	13
1.3.3 Определение контекста информационной системы сайта	14
1.3.4 Возможности системы и варианты использования	15
1.4 Обоснование выбора и определение методологии программы и методики испытаний ПМИ.....	20
ГЛАВА 2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ	27
2.1 Анализ требований и определение спецификаций.....	28
2.2 Проектирование BPMN-диаграммы бизнес-процессов	29
2.3 Составление плана базы данных	31
2.4 Описание логической и физической модели данных.....	31
2.5 Функциональное тестирование базы данных	34
ЗАКЛЮЧЕНИЕ	37
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	38
ПРИЛОЖЕНИЕ.....	39

ВВЕДЕНИЕ

Актуальность работы. В современных условиях цифровизация процессов взаимодействия между собственниками жилья и арендаторами становится ключевым фактором повышения эффективности и безопасности сделок в сфере недвижимости. Рынок краткосрочной и долгосрочной аренды характеризуется высокой динамичностью, большим объемом распределенных данных об объектах, пользователях и финансовых транзакциях. Использование разрозненных средств учета и несистематизированных каналов связи приводит к снижению прозрачности сделок, росту рисков мошенничества и усложнению управления объектами. В связи с этим возникает необходимость в проектировании и реализации централизованной реляционной базы данных, обеспечивающей единый источник достоверной информации, автоматизацию процессов бронирования и надежное хранение истории взаимодействий.

Объектом исследования является совокупность бизнес-процессов организации арендного бизнеса, связанных с поиском объектов, управлением бронированиями и анализом отзывов.

Целью работы является проектирование и разработка архитектуры базы данных в СУБД PostgreSQL для программного комплекса организации арендного бизнеса, обеспечивающей эффективное хранение данных об объектах недвижимости, управление ролями пользователей и автоматизацию учета сделок.

Задачи работы:

1. Провести анализ предметной области и выявить сущности, характерные для процессов аренды жилой недвижимости в сегменте B2C.
2. Выполнить обзор инструментальных средств и обосновать выбор СУБД PostgreSQL и среды DBeaver для реализации базы данных.
3. Выполнить моделирование бизнес-процессов системы аренды с использованием нотации BPMN с целью выявления информационных потоков и требований к структуре данных.

4. Осуществить логическое и физическое проектирование базы данных, разработав детальную ER-диаграмму.

5. Реализовать базу данных и структуру связей с помощью языка SQL в среде PostgreSQL.

6. Провести тестирование разработанной базы данных путем выполнения проверочных запросов и анализа целостности данных.

7. Провести тестирование разработанной базы данных.

ГЛАВА 1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ

1.1 Постановка задачи

В условиях активного развития цифровых сервисов в сфере недвижимости особую значимость приобретает создание эффективных онлайн-платформ для аренды жилья. Настоящая курсовая работа направлена на разработку прототипа веб-сайта, который обеспечит пользователям удобный доступ к поиску и аренде квартир.

Основные функциональные модули проекта:

1. Система управления учётными записями. Предусматривает реализацию двух ключевых сценариев: регистрацию новых пользователей и процедуру аутентификации для получения доступа к персональному разделу сайта.

2. Информационная база объектов недвижимости. Предполагает проектирование структуры хранения данных о квартирах, включающей: текстовые описания, фотоматериалы, стоимость аренды и специфические условия договора.

3. Механизм поиска с гибкой фильтрацией. Разрабатывается интерактивный интерфейс, позволяющий пользователям отбирать подходящие варианты по ряду параметров: ценовой диапазон, географическое расположение, число комнат и иным значимым характеристикам.

4. Модуль публикации объявлений. Создаётся инструментарий для собственников жилья, дающий возможность размещать, корректировать и удалять предложения об аренде.

5. Система взаимодействия участников. Внедряется функционал обратной связи: арендаторы смогут оставлять отзывы и выставлять оценки, что повысит прозрачность предложений и укрепит доверие между пользователями.

6. Пользовательский интерфейс. Проектируется адаптивный дизайн, отвечающий современным стандартам UX/UI. Интерфейс должен обеспечивать

комфортное взаимодействие с сайтом как на стационарных компьютерах, так и на мобильных устройствах.

Для реализации поставленных задач планируется:

- провести сравнительный анализ действующих платформ аналогичного назначения;
- сформировать перечень функциональных и эргономических требований на основе пользовательских ожиданий;
- разработать техническую документацию проекта;
- выполнить программную реализацию ключевых модулей;
- провести комплексное тестирование работоспособности системы.

Итоговым результатом станет функционирующий прототип веб-сайта, демонстрирующий основные сценарии взаимодействия пользователей с платформой аренды жилья.

При этом в рамках данной курсовой работы основной упор делается на разработку серверной части, включая проектирование базы данных, реализацию бизнес-логики и создание API для взаимодействия с клиентской частью. Смежные компоненты (пользовательский интерфейс, верстка страниц) разрабатываются другими участниками команды и рассматриваются в работе лишь в контексте интеграции с бэкендом.

1.2 Обзор инструментальных средств

При разработке веб-сайта по аренде квартир перед командой стояла задача выбора оптимального стека технологий, обеспечивающего надёжность, производительность и удобство разработки. В данном разделе представлен анализ альтернатив и обоснование выбранных инструментов.

1.2.1 Выбор системы управления базами данных

Для выбора оптимальной системы управления базами данных был проведен сравнительный анализ наиболее популярных реляционных решений. Результаты сравнения представлены в качестве таблицы 1.

Таблица 1

Сравнительный анализ реляционных СУБД для хранения данных системы

Критерий	PostgreSQL	MySQL	Microsoft SQL Server
Лицензия	Бесплатная	Бесплатная	Платная
Поддержка JSON	Отличная	Ограниченная	Хорошая
Сложные запросы	Отличная поддержка	Хорошая	Отличная
Надёжность	Высокая	Средняя	Высокая
Кроссплатформенность	Да	Да	Ограничена

Обоснование выбора PostgreSQL:

- бесплатная лицензия не требует дополнительных затрат;
- отличная поддержка JSON позволяет хранить гибкие структуры данных;
- поддержка сложных запросов необходима для реализации гибкого поиска по квартирам;
- надёжность и целостность данных критически важны для работы с бронированиями и платежами.

инструмент DBeaver выбран как универсальный клиент для работы с PostgreSQL благодаря бесплатности, поддержке всех необходимых функций (SQL-редактор, просмотр структуры таблиц, импорт/экспорт данных) и удобному интерфейсу.

1.2.2 Выбор сред разработки

Для выбора оптимальной системы управления базами данных был проведен сравнительный анализ наиболее популярных реляционных решений. Результаты сравнения представлены в качестве таблицы 2.

Таблица 2

Сравнительный анализ сред разработки и визуального моделирования

Инструмент	Область применения	Альтернативы	Обоснование выбора
Microsoft Visual Studio 2022	Backend (C#)	JetBrainsRider, VS Code	Встроенная поддержка ASP.NET Core, удобная отладка, профилирование кода, интеграция с Git
Visual Studio Code	Frontend (HTML, CSS, JS)	Sublime Text, WebStorm	Лёгкая, скорость работы, богатая экосистема расширений для вёрстки, встроенный терминал

Обоснование разделения: Visual Studio обеспечивает максимальную интеграцию с экосистемой .NET, что важно для разработки на C#, в то время как VS Code идеально подходит для быстрой работы с фронтенд-кодом.

1.2.3 Выбор дополнительных инструментов

Для обеспечения эффективного процесса написания кода и визуального моделирования был выполнен обзор доступных программных решений. Обоснование выбора инструментальных средств приведено в таблице 3.

Таблица 3

Сравнительный анализ инструментальных средств разработки серверной части

Инструмент	Область применения	Альтернативы	Обоснование выбора
ASP.NET Core	Фреймворк для backend	Node.js (Express), Django	Встроенная поддержка REST API, внедрение зависимостей, высокая производительность, встроенная документация
Npgsql	ORM для работы с PostgreSQL	Entity Framework Core, Dapper	Выбран по решению интегратора. Npgsql позволяет выполнять подготовленные SQL-запросы напрямую, что даёт полный контроль над выполняемым кодом и упрощает интеграцию с уже написанными запросами
Swagger	Документирование и тестирование API	Postman, Insomnia	Встроен в ASP.NET Core, автоматически генерирует документацию по коду контроллеров, позволяет тестировать API прямо в браузере

1.2.4 Выбор архитектурного подхода

В проекте реализована трёхуровневая архитектура:

1. Уровень доступа к данным – реализован интегратором в проекте Arenda.DataAccess. Содержит SQL-запросы и методы для их выполнения через Npgsql;

2. Уровень бизнес-логики – реализован в сервисах (Services/) проекта Arenda.API. Содержит валидацию, обработку данных, формирование DTO;

3. Уровень представления – контроллеры (Controllers/), обрабатывающие HTTP-запросы, проверяющие авторизацию и возвращающие ответы.

Такой подход обеспечивает разделение ответственности, упрощает тестирование и поддержку кода.

На основе проведенного анализа был сформирован итоговый перечень технологий, необходимых для реализации всех функциональных модулей системы. Состав технологического стека разработки программного комплекса представлен в качестве таблицы 4.

Таблица 4

Состав технологического стека разработки программного комплекса

Категория	Выбранный инструмент
Backend	C# / ASP.NET Core
Frontend	HTML, CSS, JavaScript
База данных	PostgreSQL
Клиент БД	DBeaver
ORM	Npgsql
IDE для backend	Microsoft Visual Studio 2022
Редактор для frontend	Visual Studio Code
Документирование API	Swagger (встроен в ASP.NET Core)
Контроль версий	Git

1.3 Анализ предметной области и определение возможностей системы

1.3.1 Анализ конкурентов

Был выполнен детальный анализ текущего состояния рынка цифровых сервисов в сфере недвижимости. Изучение существующих решений позволяет не только выявить их сильные и слабые стороны, но и четко определить рыночную нишу для проектируемого продукта, а также сформировать уникальное ценностное предложение, отвечающее актуальным запросам пользователей.

Основные конкуренты:

- классические доски объявлений (Авито, Циан): обладают огромной базой объектов и широкой аудиторией. Однако их основная модель долгое время строилась как "доска объявлений", где платформа минимально участвовала в сделке, что создавало риски мошенничества. В последнее время они внедряют онлайн-бронирование и проверку, но этот функционал часто распространяется не на все объекты;
- сервисы краткосрочной аренды (Airbnb, Суточно.ру): сфокусированы на посуточной аренде и туристическом размещении. Механики долгосрочной аренды (на месяцы) проработаны слабее;
- управляющие сервисы полного цикла (Colife, Pik-arenda): предлагают аренду с сервисом, но часто работают только с собственным пулом квартир, выступая в роли управляющей компании, а не открытой площадки.

В контексте вышеизложенного, для определения конкретных конкурентных преимуществ и ключевых функциональных особенностей проектируемой B2C платформы был проведен сравнительный анализ. Результаты сопоставления предлагаемого решения с существующими на рынке аналогами представлены в качестве таблицы 5.

Сравнительный анализ архитектурных паттернов проектирования
программного обеспечения

Критерий	Конкуренты (Авито, Циан)	Наше предложение
Модель участия	Площадка для встречи сторон	Активный гарант сделки
Безопасность	Частичная проверка объектов	Сквозная верификация пользователей, обязательная модерация объявлений, логирование всех действий
Автоматизация	Ручное общение в чатах	Автоматизированные бизнес-процессы (статусы заявок, уведомления, модерация)
Ролевая модель	Гость / Владелец	5 ролей (Пользователь, Арендодатель, Модератор, Админ, Поддержка)
Качество контента	Высокий риск спама и дубликатов	Предварительная модерация всех объявлений перед публикацией
Интеграция	Закрытые экосистемы	Открытый API для интеграции

Таким образом, разрабатываемый продукт предлагает сбалансированное решение: он сочетает открытость площадки (как у агрегаторов) с глубиной контроля и автоматизации сделок (как у управляющих сервисов), делая особый упор на качество контента через модерацию, надёжную бэкенд-логику и безопасность данных.

1.3.2 Уникальность разрабатываемого продукта

Современный рынок аренды жилья в России характеризуется высокой динамикой цифровизации и усилением конкуренции между платформами. Крупные игроки, такие как Циан, Авито, Яндекс.Аренда, предлагают пользователям развитые сервисы: онлайн-бронирование, проверку объектов, страхование сделок. Однако многие существующие решения либо ориентированы на краткосрочную аренду, либо представляют собой классические доски объявлений с ограниченным участием платформы в обеспечении безопасности сделок.

Уникальность разрабатываемого продукта заключается в комплексном подходе к автоматизации арендного бизнеса с акцентом на прозрачность, безопасность и удобство для всех участников процесса. В отличие от простых агрегаторов, проектируемая платформа предлагает полноценную экосистему, где каждый этап – от поиска квартиры до заключения договора и оплаты – поддерживается серверной логикой.

Ключевые уникальные характеристики:

1. Разделение пользовательских ролей на уровне бэкенда. Система чётко разграничивает права доступа для пяти категорий: Пользователь (арендатор), Арендодатель, Модератор, Администратор, Служба поддержки. Для каждой роли реализованы специфические сценарии, что повышает безопасность данных и исключает конфликт операций;

2. Автоматизация бизнес-процессов. В отличие от платформ, где коммуникация происходит в ручном режиме, разрабатываемый бэкенд автоматизирует ключевые процессы: обработку заявок на аренду, уведомление сторон об изменении статусов, модерацию объявлений;

3. API-ориентированная архитектура. Серверная часть проектируется как независимый модуль с открытым API, что обеспечивает гибкость интеграции с любым фронтендом (веб, мобильное приложение) и упрощает дальнейшее масштабирование системы.

1.3.3 Определение контекста информационной системы сайта

Диаграмма контекста отображает разрабатываемую систему как единое целое и её взаимодействие с внешними сущностями. Система рассматривается как «черный ящик» – показываются только её границы и то, что находится за их пределами.

Главная функция системы: «Управление процессом аренды квартир».

Внешние сущности (факторы):

1. Пользователь (арендатор) – лицо, ищущее квартиру для аренды;
2. Арендодатель – собственник жилья, размещающий объявления;
3. Модератор – сотрудник платформы, проверяющий объявления на соответствие правилам, принимающий решение о публикации или отклонении, обрабатывающий жалобы;
4. Администратор – технический специалист, управляющий системой и пользователями;
5. Служба поддержки – операторы, обрабатывающие обращения пользователей;
6. Платежная система – внешний сервис для обработки финансовых транзакций;
7. Сторонние API-сервисы – внешние сервисы (например, карты, погода, геолокация).

Входные данные:

- данные регистрации и авторизации;
- запросы на поиск квартиры;
- данные о новых объявлениях;
- запросы на бронирование;
- решения по модерации (одобрение/отклонение);
- жалобы на объявления;
- платежная информация;
- заявки в поддержку;

- выходные данные;
- результаты поиска (список квартир);
- подтверждение регистрации и действий;
- уведомления (email/push);
- решение модератора (публикация/отклонение);
- информация для платежной системы;
- аналитические отчеты;
- ответы службы поддержки.

Управление:

- законодательство РФ (о персональных данных, об аренде);
- политика безопасности платформы;
- правила модерации (требования к объявлениям);
- бизнес-правила (условия аренды, бронирования).

Механизм:

- серверное оборудование;
- СУБД PostgreSQL;
- веб-сервер;
- разработанное ПО на ASP.NET Core.

Для определения границ проектируемой системы и её взаимодействия с внешними сущностями была разработана контекстная диаграмма. Общая схема информационных потоков и границ программного комплекса представлена в качестве рисунка 1.

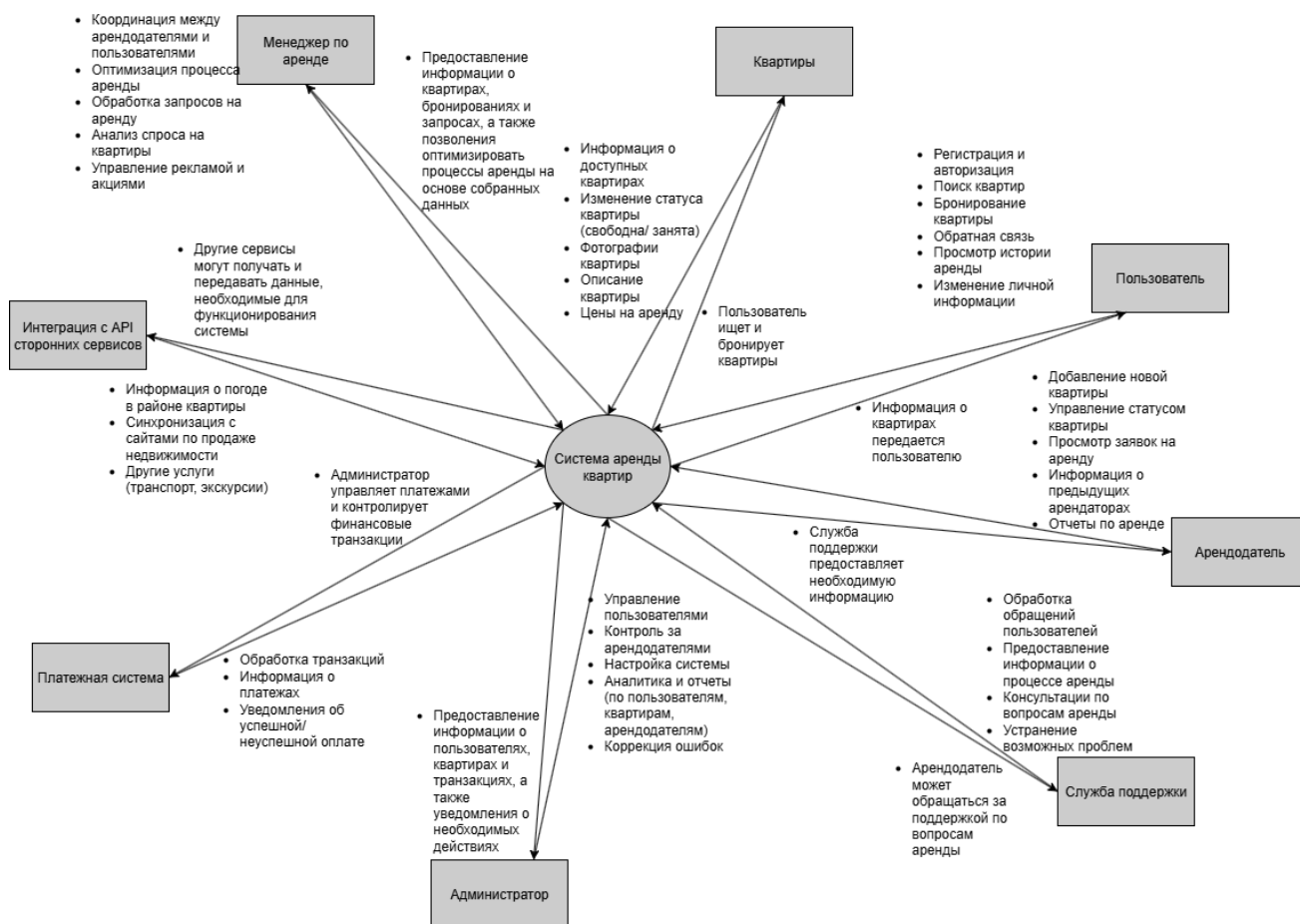


Рис. 1 «Контекстная диаграмма системы аренды квартир»

1.3.4 Возможности системы и варианты использования

Use Case (диаграмма вариантов использования) описывает функциональные требования к системе с точки зрения пользователя. Она показывает, какие действия могут выполнять различные акторы (роли) при взаимодействии с системой.

На основе анализа предметной области и выделенных ролей были разработаны диаграммы вариантов использования для каждой категории пользователей платформы аренды квартир.

Для детального описания функциональных возможностей системы необходимо рассмотреть сценарии взаимодействия для каждой роли пользователей. Набор функций, доступных для управления системой и контроля её параметров, представлен на рисунке 2.

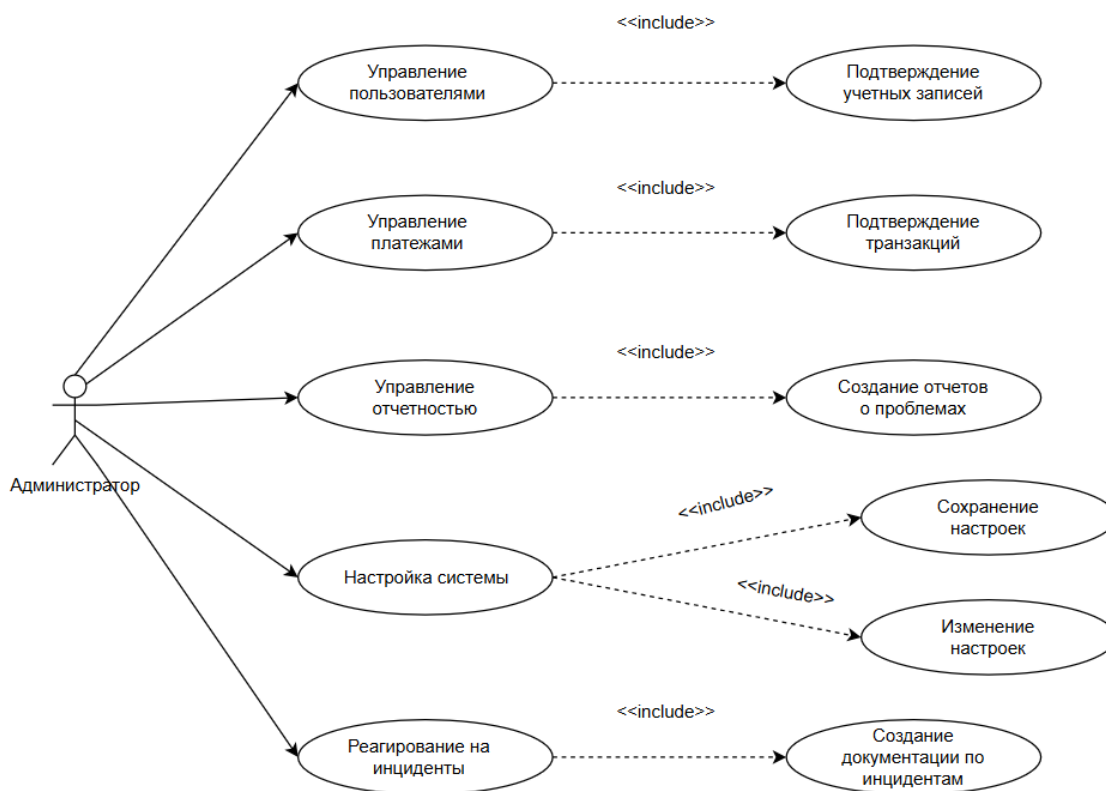


Рис. 2 «Администратор»

Администратор управляет техническим состоянием платформы и правами доступа. Основные варианты использования:

- управление пользователями – добавление, редактирование, блокировка, удаление пользователей, подтверждение учетных записей;
- управление платежами – просмотр истории платежей, подтверждение и отмена транзакций;
- управление отчетностью – создание отчетов о проблемах, статистике использования сайта и финансовых показателях;
- настройка системы – изменение параметров работы платформы;
- реагирование на инциденты – обработка сбоев и нарушений безопасности, создание документации по инцидентам.

Основным поставщиком контента в системе является арендодатель, чьи возможности по управлению объектами недвижимости отражены на рисунке 3.

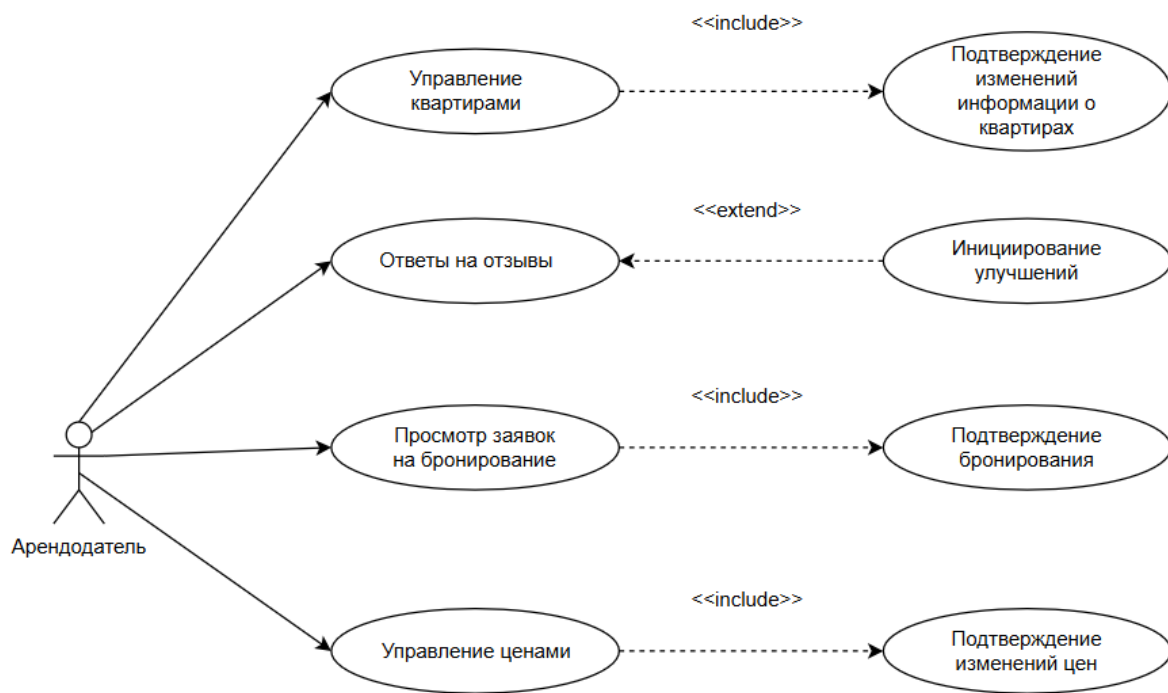


Рис. 3 «Арендодатель»

Арендодатель – владелец квартиры, заинтересованный в сдаче жилья.

Основные варианты использования:

- управление квартирами – добавление, редактирование, удаление квартир (включает подтверждение изменений информации о квартирах);
- ответы на отзывы – просмотр и ответы на отзывы (расширяется инициированием улучшений);
- просмотр заявок на бронирование – просмотр входящих заявок (включает подтверждение или отклонение бронирования);
- управление ценами – изменение цен на квартиры (включает подтверждение изменений цен).

В целях обеспечения безопасности сделок и чистоты данных в системе предусмотрена роль модератора. Перечень его функциональных задач представлен в качестве рисунка 4.

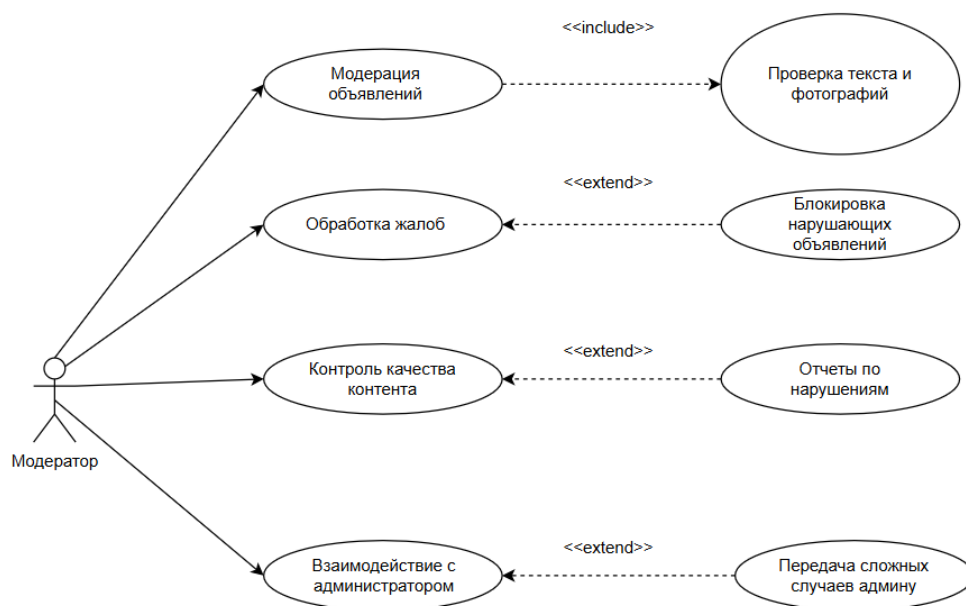


Рис. 4 «Модератор»

Ключевым потребителем сервиса является конечный арендатор (пользователь). Сценарии его взаимодействия с интерфейсом платформы и процессами поиска жилья показаны на рисунке 5.

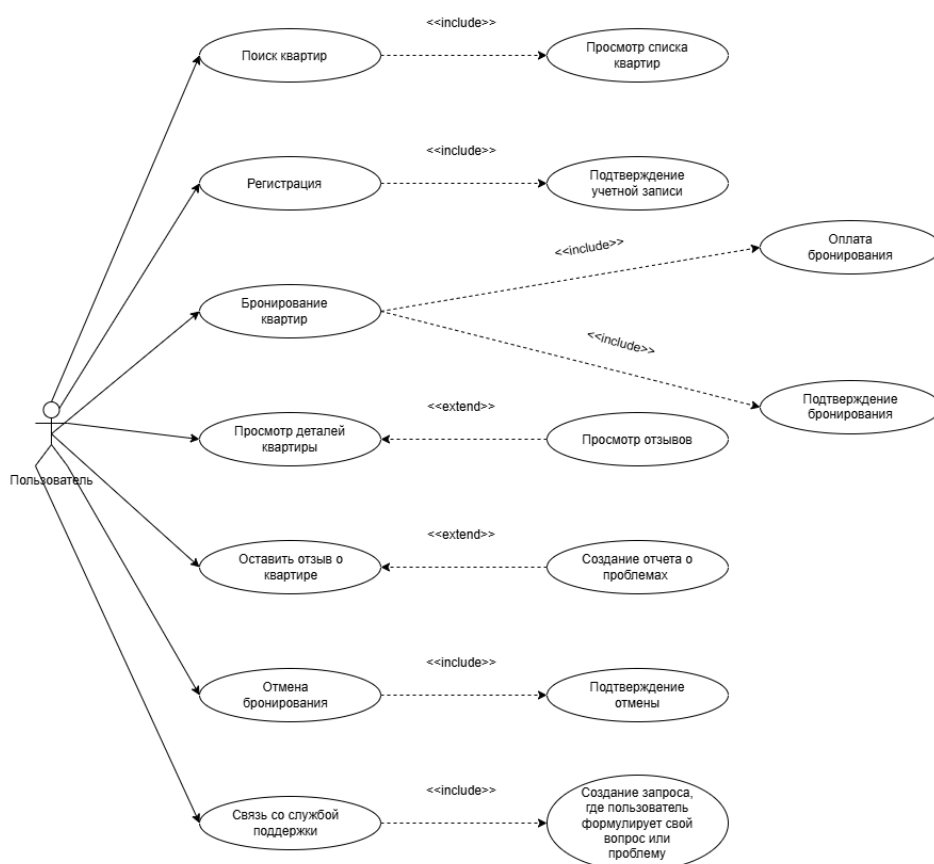


Рис. 5 «Пользователь»

Для оперативного решения спорных ситуаций и технической помощи клиентам в архитектуру системы заложена роль службы поддержки. Функциональные возможности данной роли отражены на рисунке 6.

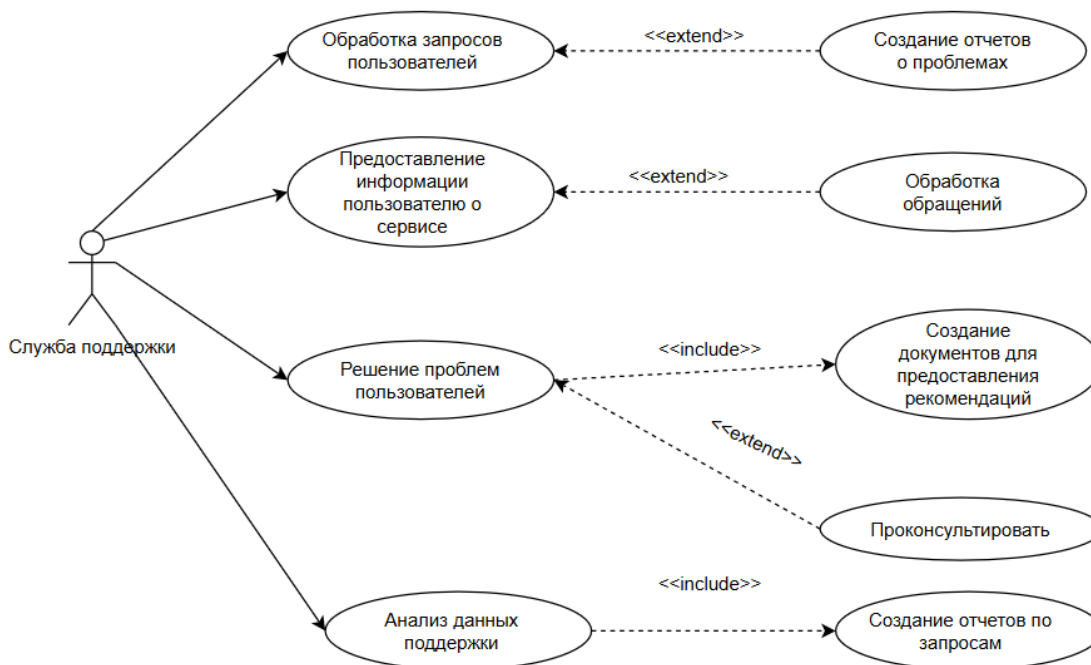


Рис. 6 «Служба поддержки»

1.4 Обоснование выбора и определение методологии программы и методики испытаний ПМИ

1. Определение методологии испытаний. Выбор методологии программы и методики испытаний является ключевым этапом разработки информационной системы, так как он определяет системный подход к подтверждению работоспособности программного продукта и его соответствия требованиям технического задания. В рамках данной работы методология испытаний базируется на положениях государственных стандартов, в частности ГОСТ 19.301-79 «Программа и методика испытаний», что обеспечивает структурированность и объективность процесса верификации.

Методология испытаний — это совокупность правил, методов и инструментов, необходимых для контроля качества ИС. Она показывает, что программа тестируется не «наугад», а системно, подтверждая реализацию всех функциональных возможностей системы в заданных условиях эксплуатации.

2. Обоснование выбора видов испытаний. Для обеспечения всесторонней проверки разрабатываемой информационной системы выбраны следующие уровни проверки:

- автономные испытания: проверка отдельных модулей сразу после их написания. Это позволяет локализовать ошибки на ранней стадии;
- комплексные испытания: проверка того, как программные компоненты работают вместе и как система взаимодействует с базой данных;
- приемо-сдаточные испытания: итоговая проверка всех функций на соответствие техническому заданию перед сдачей проекта.

3. Выбор стратегии тестирования. В качестве основной стратегии контроля функциональности выбран метод «белого ящика». Данный выбор обусловлен тем, что объектом разработки является серверная часть, где критически важно обеспечить корректность выполнения алгоритмов, обработки данных и взаимодействия с СУБД на уровне программного кода.

Метод «белого ящика» позволяет:

- проверить все пути выполнения бизнес-логики в сервисах, включая обработку исключений и граничные условия;
- гарантировать корректность формирования и выполнения SQL-запросов через библиотеку Npgsql в слое доступа к данным;
- оценить полноту покрытия кода тестами, что минимизирует риск появления логических ошибок в архитектуре системы;
- верифицировать соблюдение правил целостности данных при выполнении сложных операций, таких как бронирование объектов или расчет стоимости.

Для малого масштаба ИС выбрано ручное тестирование по сценариям, так как внедрение автотестов экономически нецелесообразно.

4. СУБД PostgreSQL. Выбор данной системы управления базами данных обусловлен её объектно-реляционной архитектурой, высокой надежностью и

поддержкой сложных типов данных. Для системы аренды недвижимости в распределенной сети PostgreSQL является оптимальным решением благодаря:

- Строгому соблюдению принципов ACID, что критически важно для обработки финансовых транзакций (предоплаты) и исключения двойного бронирования.
- Высокой производительности при работе с большими объемами данных и сложными связями между таблицами (страны, регионы, города, карточки квартир).
- Наличие механизмов контроля целостности данных (Foreign Keys, Check Constraints), что гарантирует корректность связей между пользователями, их ролями и объектами недвижимости.

Для подтверждения корректности работы реализованного функционала и соответствия системы предъявляемым требованиям, был разработан план тестирования ключевых модулей. Программа и методика испытаний представлены в таблице 6.

Таблица 6

Программа и методика испытаний базы данных

Объект испытания	Проверяемая функция / Требование	Метод испытания	Ожидаемый результат
1	2	3	4
Структура таблиц и первичные ключи	Корректность создания сущностей БД и типов данных	Выполнение DDL-скрипта CREATE TABLE. Попытка вставить новую запись с уже существующим Primary Key.	Таблицы успешно созданы в СУБД. При попытке вставки дубликата база данных выдает ошибку нарушения уникальности ключа.
Ссылочная целостность (Внешние ключи)	Поддержка строгих связей между таблицами (например, между bronirovaniye и kartochka_kvartiry)	Попытка добавить запись о бронировании, указав несуществующий ID квартиры.	СУБД блокирует операцию. Данные остаются консистентными, запись не добавляется.

1	2	3	4
Ограничения значений (CHECK, NOT NULL)	Запрет на ввод некорректных данных (например, пустых полей или отрицательной стоимости)	Выполнение INSERT-запроса с NULL в обязательном поле или отрицательным значением суммы предоплаты.	СУБД отклоняет транзакцию, возвращая ошибку нарушения ограничения NOT NULL или CHECK.
Операции манипулирования данными	Корректное добавление, обновление и каскадное удаление записей	Выполнение скриптов INSERT, UPDATE, DELETE для тестового набора данных по арендаторам и жилью.	Данные успешно добавлены, изменены и удалены в соответствующих таблицах без потери ссылочной целостности.

Результаты выполнения сценариев тестирования

Сценарий 1. Структура таблиц и первичные ключи

Метод испытания: выполнение DDL-скрипта CREATE TABLE. Попытка вставить новую запись с уже существующим Primary Key.

Ожидаемый результат: таблицы успешно созданы в СУБД. При попытке вставки дубликата база данных выдает ошибку нарушения уникальности ключа.

SQL-запрос для тестирования представлены на рисунке 7:

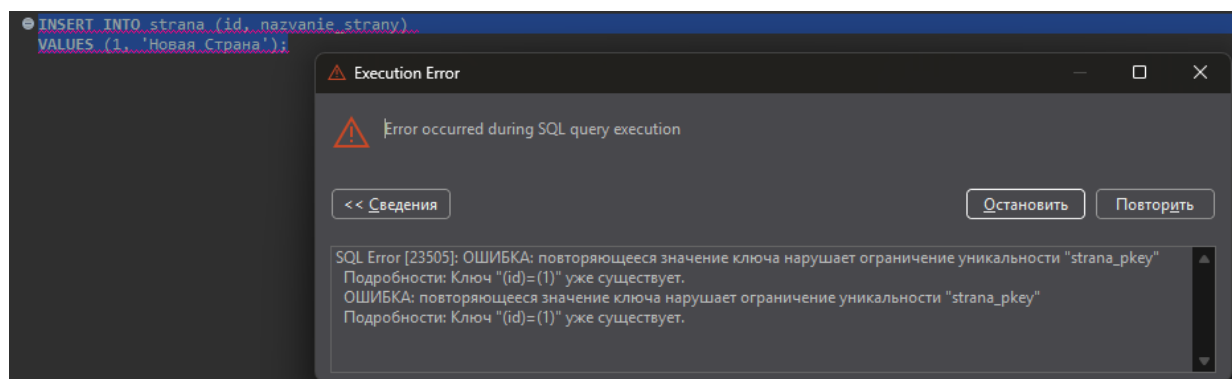


Рис. 7 «Выполнение DDL-скрипта»

СУБД блокирует транзакцию и выводит сообщение об ошибке: ОШИБКА: повторяющееся значение ключа нарушает ограничение уникальности "strana_pkey". Ожидаемый результат достигнут.

Сценарий 2. Ссылочная целостность (Внешние ключи)

Метод испытания: попытка добавить запись о бронировании, указав несуществующий ID квартиры.

Ожидаемый результат: СУБД блокирует операцию. Данные остаются консистентными, запись не добавляется.

SQL-запрос для тестирования представлены на рисунке 8:

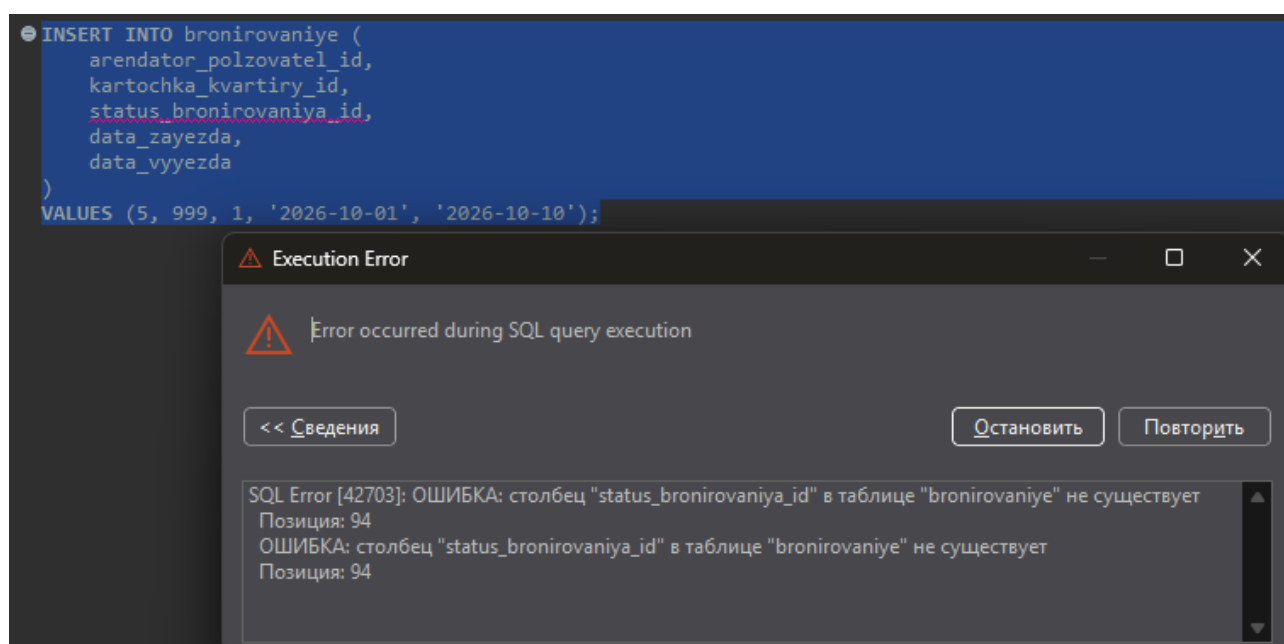


Рис. 8 «Добавление записи о бронировании»

Результат: база данных прерывает операцию с ошибкой нарушения внешнего ключа: ОШИБКА: столбец "status_bronirovaniya_id" в таблице "bronirovaniye" не существует. Ссылочная целостность подтверждена.

Сценарий 3. Ограничения значений (CHECK, NOT NULL)

Метод испытания: выполнение INSERT-запроса с NULL в обязательном поле или отрицательным значением суммы предоплаты.

Ожидаемый результат: СУБД отклоняет транзакцию, возвращая ошибку нарушения ограничения NOT NULL или CHECK.

SQL-запрос для тестирования представлены на рисунке 9:

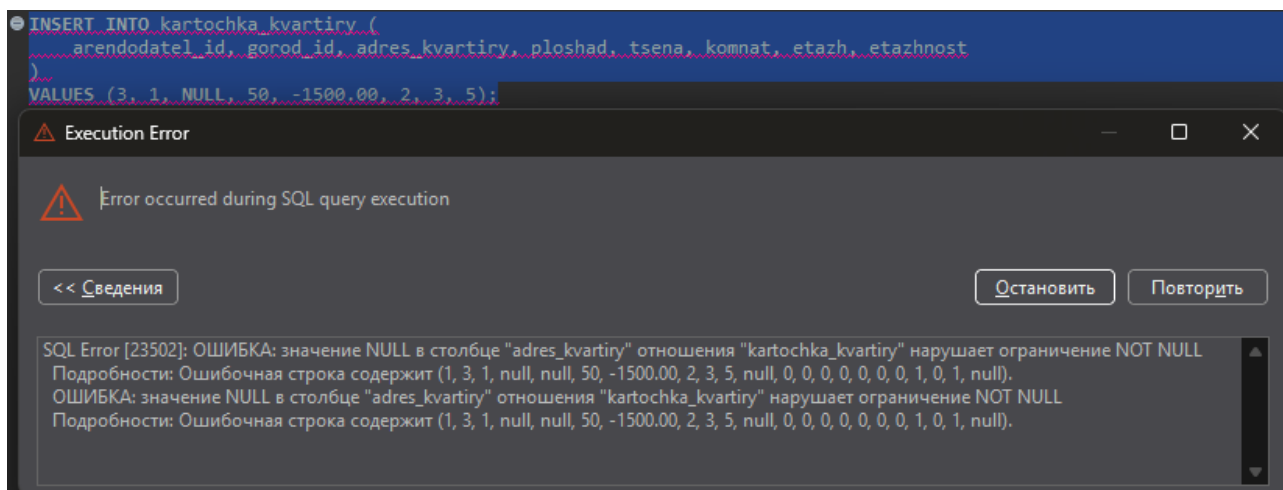


Рис. 9 «Выполнение INSERT-запроса с NULL»

Результат: транзакция отклонена. СУБД выдает предупреждение об ошибке: ОШИБКА: значение NULL в столбце "adres_kvartiry" отношения "kartochka_kvartiry" нарушает ограничения NOT NULL. Ограничения работают корректно.

Сценарий 4. Операции манипулирования данными (DML)

Метод испытания: Выполнение скриптов INSERT, UPDATE, DELETE для тестового набора данных по арендаторам и жилью.

Ожидаемый результат: Данные успешно добавлены, изменены и удалены в соответствующих таблицах без потери ссылочной целостности.

SQL-запросы для тестирования представлены на рисунке 10:

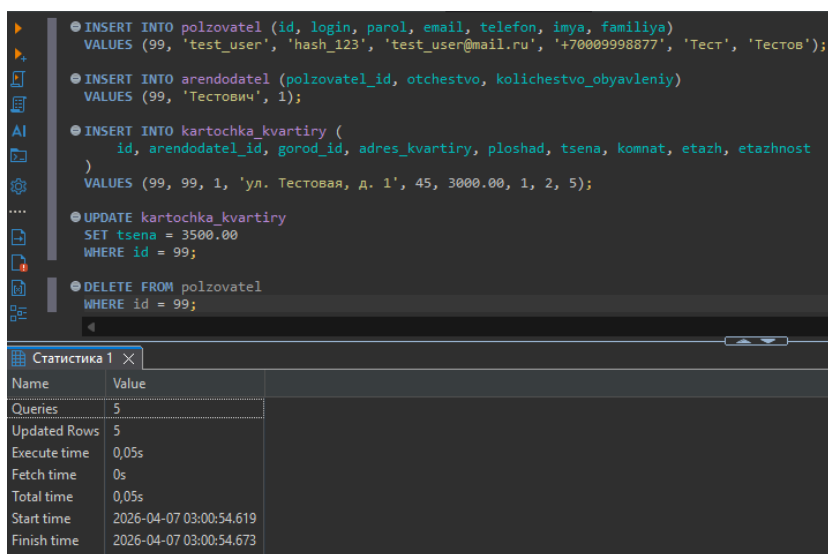


Рис. 10 «Выполнение скриптов INSERT, UPDATE, DELETE»

Результат: все операции прошли успешно. Скрипт DELETE корректно удалил пользователя (id=99) и, благодаря настроенным правилам каскадного удаления, автоматически очистил зависимые записи в таблицах arendodatel и kartochka_kvartiry. Данные базы остались полностью консистентными.

ГЛАВА 2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ

2.1 Анализ требований и определение спецификаций

В данном разделе осуществляется подробное описание функциональных возможностей системы и формулируются основные требования к программному продукту. Исходя из спроектированной структуры данных ключевым требованием является реализация полноценного управления пользователями с четким разграничением ролей администраторов, модераторов, арендаторов и арендодателей при сохранении уникальных наборов данных для каждой категории. Система обеспечивает строгую географическую привязку объектов недвижимости через поддержку иерархической структуры адресов от страны и региона до конкретного города. Важной частью функционала выступает управление контентом и объявлениями, что подразумевает создание детальных карточек объектов с указанием технических характеристик правил проживания и интеграцией мультимедийных файлов. Основной бизнес-процесс реализует полный цикл взаимодействия в рамках схемы от поиска и бронирования до подтверждения предоплаты. Для обеспечения качества сервиса внедряется система отзывов с обязательной премодерацией и классификатором оценок по пятибалльной шкале, а также модуль обратной связи и техподдержки для фиксации инцидентов, привязанных к конкретным бронированиям или объектам.

Для обеспечения бесперебойной работы платформы аренды недвижимости, сформулированные требования детализированы по нескольким ключевым подсистемам. Подсистема управления доступом должна гарантировать строгую изоляцию данных на программном уровне. Модераторы получают доступ исключительно к пулу необработанных отзывов и спорных ситуаций, в то время как администраторы наделяются правами управления системными справочниками. Для арендодателей и арендаторов предусмотрены специализированные личные кабинеты, учитывающие специфику их

взаимодействия с платформой: отслеживание количества размещенных объявлений для одних и истории бронирований для других.

Подсистема управления объектами недвижимости требует реализации гибких поисковых механизмов. Пользователь должен иметь возможность осуществлять поиск не только по базовым параметрам, таким как город, цена или количество комнат, но и по специфическим условиям, включая допустимость проживания с детьми или животными. Географическая структура базы данных призвана исключить ошибки текстового ввода пользователями и жестко стандартизировать поисковые запросы на основе связанных справочников.

Транзакционная подсистема отвечает за ядро бизнес-логики приложения. Главным техническим требованием здесь является реализация надежного механизма блокировки пересекающихся дат на уровне базы данных или сервисного слоя, чтобы полностью исключить возможность овербукинга. Этот процесс тесно интегрирован с финансовым модулем, который фиксирует факт внесения предоплаты, выбранный способ оплаты и актуальный статус транзакции, обеспечивая возможность последующего возврата средств при отмене брони.

Наконец, подсистема контроля качества должна обеспечивать высокий уровень доверия к сервису. Публикация отзывов включает обязательный этап проверки, что защищает платформу от некорректного контента. Модуль технической поддержки построен по принципу системы тикетов: каждое пользовательское обращение жестко привязывается к идентификаторам конкретных бронирований или квартир, что предоставляет операторам полный контекст проблемы для ее оперативного решения.

2.2 Проектирование BPMN-диаграммы бизнес-процессов

Для формализации логики взаимодействия участников системы и детального описания последовательности выполняемых операций была разработана диаграмма в нотации BPMN. Это позволило визуализировать информационные потоки и определить точки принятия решений в рамках основных сценариев платформы. Визуализация спроектированных бизнес-процессов представлена на рисунке 11.

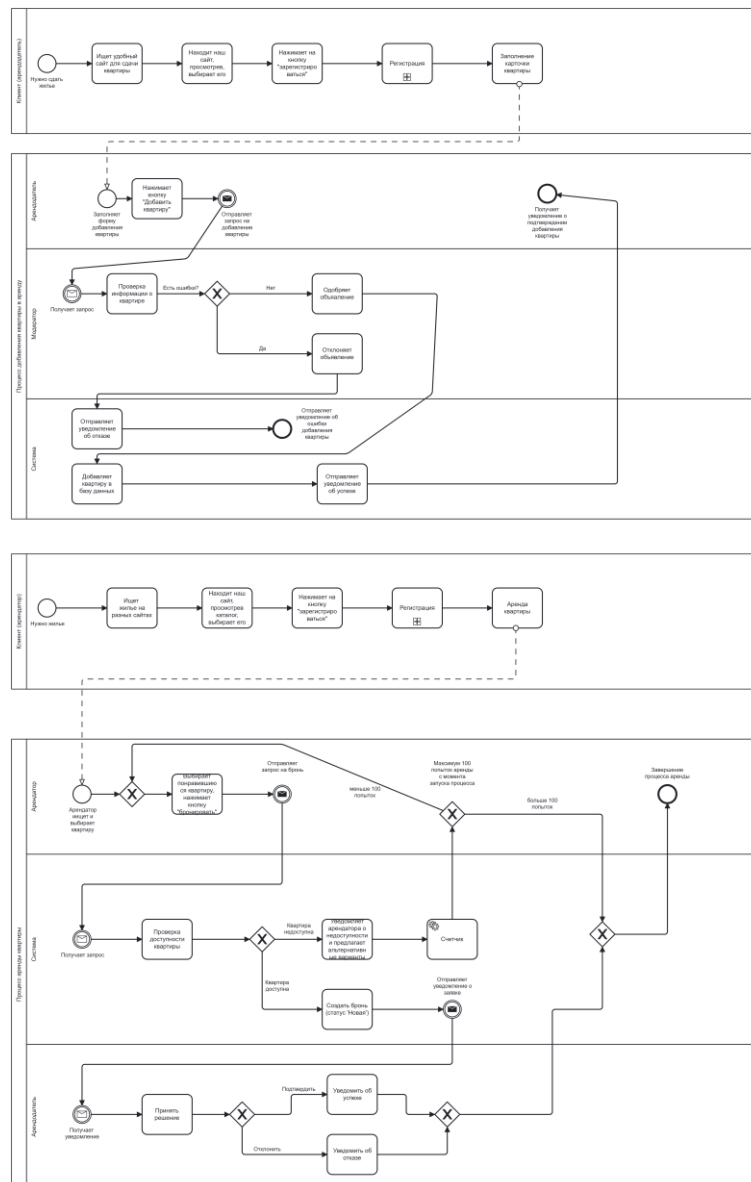


Рис. 11 «BPMN-диаграмма»

На представленной диаграмме BPMN детально визуализированы два ключевых жизненных цикла платформы: процесс публикации объявления и

процесс аренды недвижимости. Каждый из них разделен на зоны ответственности, что позволяет проследить взаимодействие между пользователями, модераторами и автоматизированными алгоритмами серверной части.

Процесс добавления квартиры в аренду инициируется арендодателем, который после регистрации в системе переходит к заполнению детальной карточки объекта. После нажатия кнопки публикации запрос поступает в зону ответственности модератора. Здесь реализована критическая развилка: если в данных обнаружены ошибки, модератор отклоняет объявление, а система автоматически формирует и отправляет пользователю уведомление об отказе с указанием причин. В случае успешной проверки модератор одобряет публикацию, после чего система вносит запись в базу данных и уведомляет арендодателя о том, что его квартира стала доступна в общем каталоге.

Процесс аренды квартиры представлен в виде описания взаимодействия арендатора с платформой, которое начинается с поиска и выбора подходящего варианта. После нажатия кнопки «бронировать» система инициирует автоматическую проверку доступности жилья на выбранные даты. В случае занятости объекта система уведомляет об этом клиента, предлагает альтернативные варианты и включает логику счетчика попыток, ограничивая их число до 100 для предотвращения бесконечных циклов.

Если же квартира доступна, система переводит бронирование в статус «Новая» и отправляет уведомление арендодателю. В завершающем этапе арендодатель может подтвердить или отклонить бронь. В зависимости от выбора система уведомляет арендатора об успешном завершении сделки либо об отказе, после чего бизнес-процесс считается закрытым.

Эта визуализация подтверждает, что архитектура системы предусматривает не только линейное прохождение задач, но и обработку исключительных ситуаций, таких как ошибки модерации или занятость выбранных объектов.

2.3 Составление плана базы данных

Здесь стоит указать, что база данных спроектирована по принципу *нормализации* (до 3-й нормальной формы), чтобы избежать дублирования данных. Основные логические блоки:

- блок «Локация»: strana, region, gorod, gorodstrana;
- блок «Субъекты»: polzovatel и его расширения через связи 1:1 (administrator, moderator и т.д.);
- блок «Объект аренды»: kartochka_kvartiry, photo_kvartiry, справочники условий;
- блок «Сделка»: bronirovaniye, predoplata, status_predoplaty;
- блок «Сервис»: otzivy, zaprosi_podderzhki.

2.4 Описание логической и физической модели данных

Логическая модель представляет собой архитектурное решение, абстрагированное от конкретной реализации в СУБД. Она ориентирована на бизнес-логику процесса аренды. В данной работе модель построена по принципу нормализации до 3-й нормальной формы, что исключает избыточность.

Проектирование базы данных является фундаментом серверной части системы, так как от структуры таблиц напрямую зависит производительность запросов и масштабируемость приложения. Спроектированная структура базы данных представлена в виде ER-диаграммы на рисунке 12.

– целостность связей (применены правила ON DELETE CASCADE для автоматического удаления зависимых записей и ON DELETE RESTRICT для защиты критически важных справочников);

– оптимизация (созданы индексы (CREATE INDEX) для всех полей, участвующих в поиске и фильтрации: цены, даты бронирования, логинов и номеров телефонов);

– типы данных (использованы специализированные типы, такие как DECIMAL(10,2) для финансовых расчетов, BIT для флагов одобрения и TIMESTAMP для фиксации точного времени транзакций).

Матрица данных на рисунке 13 является наглядным инструментом для проектирования баз данных и определения прав доступа к информации в системе аренды жилья.

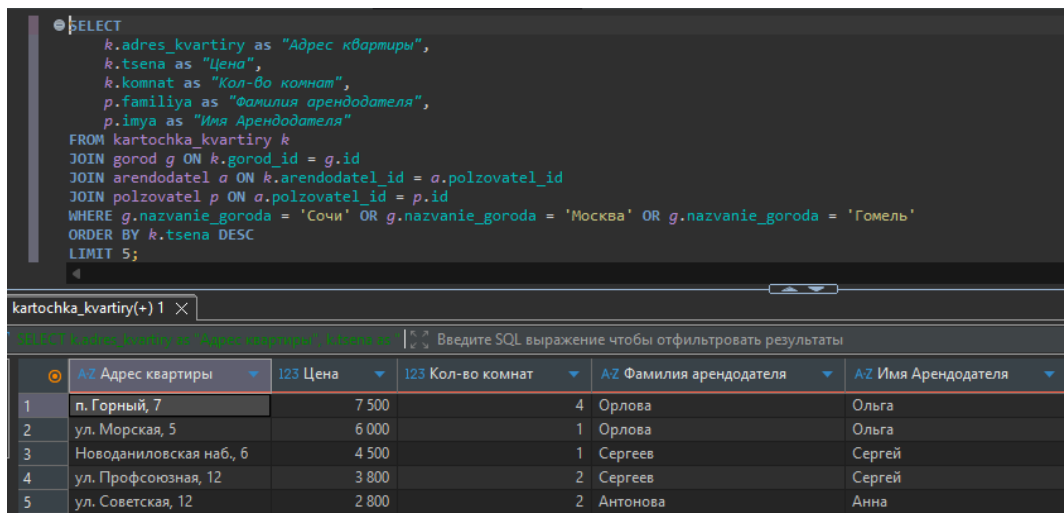
	A	B	C	D	E	F	G
1	Атрибуты\Сущности	Пользователь	Квартира	Фото квартиры	Бронирование	Отзыв	Город
2	Уникальный идентификатор (id)	+	+	+	+	+	+
3	ID пользователя (FK)		+		+	+	
4	ID квартиры (FK)			+	+	+	
5	ID города (FK)		+				
6	Логин/Email	+					
7	Пароль (хеш)	+					
8	Имя	+					
9	Фамилия	+					
10	Отчество	+					
11	Телефон	+					
12	Аватар	+					
13	Роль	+					
14	Название города						+
15	Адрес квартиры		+				
16	Цена		+				
17	Количество комнат		+				
18	Площадь		+				
19	Описание		+				
20	Удобства		+				
21	Путь к файлу			+			
22	Порядок сортировки			+			
23	Дата заезда				+		
24	Дата выезда				+		
25	Количество гостей				+		
26	Статус бронирования				+		
27	Оценка					+	
28	Текст отзыва					+	
29	Статус модерации отзыва					+	

Рис. 13 «Матрица данных»

2.5 Функциональное тестирование базы данных

В данном разделе приводятся примеры SQL-запросов, демонстрирующие корректную работу связей между таблицами и возможность извлечения сложной аналитики.

Одним из ключевых требований к разрабатываемой базе данных является возможность оперативного извлечения аналитической информации для оценки состояния рынка. В рамках реализации данного функционала была выполнена выборка наиболее дорогостоящих объектов недвижимости. Результат запроса, отображающий топ-5 самых дорогих квартир в Москве, Гомеле и Сочи с информацией об их владельцах, представлен на рисунке 14.



```
SELECT
  k.adres_kvartiry as "Адрес квартиры",
  k.tsena as "Цена",
  k.komnat as "Кол-во комнат",
  p.familiya as "Фамилия арендодателя",
  p.imya as "Имя Арендодателя"
FROM kartochka_kvartiry k
JOIN gorod g ON k.gorod_id = g.id
JOIN arendodatel a ON k.arendodatel_id = a.polzovatel_id
JOIN polzovatel p ON a.polzovatel_id = p.id
WHERE g.nazvanie_goroda = 'Сочи' OR g.nazvanie_goroda = 'Москва' OR g.nazvanie_goroda = 'Гомель'
ORDER BY k.tsena DESC
LIMIT 5;
```

	AZ Адрес квартиры	123 Цена	123 Кол-во комнат	AZ Фамилия арендодателя	AZ Имя Арендодателя
1	п. Горный, 7	7 500	4	Орлова	Ольга
2	ул. Морская, 5	6 000	1	Орлова	Ольга
3	Новоданиловская наб., 6	4 500	1	Сергеев	Сергей
4	ул. Профсоюзная, 12	3 800	2	Сергеев	Сергей
5	ул. Советская, 12	2 800	2	Антонова	Анна

Рис. 14 «Топ-5 самых дорогих квартир»

Для мониторинга вовлеченности клиентской базы и сегментации пользователей в системе предусмотрено формирование статистических данных. Ранжирование пользователей по количеству совершенных операций и частоте взаимодействий с платформой отражено в рейтинге активности, представленном на рисунке 15.

<pre> SELECT p.imya ' ' p.familiya AS "Имя и Фамилия клиента", p.email as Почта, COUNT(b.id) AS "Количество бронирований" FROM arendator a JOIN polzovatel p ON a.polzovatel_id = p.id JOIN bronirovaniye b ON a.polzovatel_id = b.arendator_polzovatel_id GROUP BY p.id, p.imya, p.familiya, p.email ORDER BY "Количество бронирований" DESC; </pre>			
olzovatel 1 X SELECT p.imya ' ' p.familiya AS "Имя и Фамилия кли Введите SQL выражение чтобы отфильтровать			
	AZ Имя и Фамилия клиента	AZ Почта	123 Количество бронирований
1	Мария Морозова	tenant2@mail.ru	3
2	Алексей Алексеев	tenant1@mail.ru	3
3	Павел Павлов	tenant4@yandex.ru	2
4	Дмитрий Дмитриев	tenant3@gmail.com	2

Рис. 15 «Рейтинг активности»

В целях обеспечения контроля качества предоставляемых услуг и своевременного реагирования на проблемные ситуации был реализован механизм фильтрации обратной связи. Инструментарий системы позволяет формировать отчеты по жалобам и негативному опыту пользователей. Пример выгрузки промодерированных отзывов с низким рейтингом для анализа службой поддержки приведен на рисунке 16.

<pre> SELECT k.adres_kvartiry AS "Адрес квартиры", r.otsenka AS Рейтинг, o.tekst_otzyva AS "Текст отзыва", o.date AS "Дата отзыва" FROM otzivy o JOIN reyting_klassifikator r ON o.reyting_klassifikator_id = r.id JOIN bronirovaniye b ON o.bronirovaniye_id = b.id JOIN kartochka_kvartiry k ON b.kartochka_kvartiry_id = k.id WHERE o.odobreno = 8*1 AND r.otsenka >= 4 ORDER BY r.otsenka ASC; </pre>			
kartochka_kvartiry(+1) X SELECT k.adres_kvartiry AS "Адрес квартиры", r.otsenka Введите SQL выражение чтобы отфильтровать результаты			
	AZ Адрес квартиры	123 Рейтинг	AZ Отзыв
1	п. Горный, 7	4	Хороший дом, тихий район, есть всё для комфортного проживания.
2	ул. Морская, 5	4	Понравилось, хороший вид из окна, всё чисто.
3	Новоданиловская наб., 6	4	Хорошая студия, удобное расположение. Немного шумновато из-за стройки рядом.
4	пр. Независимости, 30	5	Всё отлично, чисто, уютно, рекомендую!
5	ул. Кремлёвская, 10	5	Шикарная квартира, отличный ремонт, всё новое.
6	ул. Баумана, 15	5	Супер! Всё понравилось, обязательно вернёмся!
7	ул. Профсоюзная, 12	5	Отличная квартира, всё чисто, есть всё необходимое. Хозяин приветливый.

Рис. 15 «Выгрузка отзывов»

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была полностью достигнута поставленная цель – спроектирована и программно реализована база данных в СУБД PostgreSQL, предназначенная для автоматизации бизнес-процессов аренды недвижимости. Данная разработка позволила создать надежную информационную основу для взаимодействия арендодателей и арендаторов в рамках единой b2c-платформы.

Для реализации проекта был последовательно решен ряд ключевых задач. На подготовительном этапе проведен детальный анализ предметной области, который позволил выделить основные сущности системы и специфику их взаимодействия. Обоснование выбора СУБД PostgreSQL и среды DBeaver подтвердило их эффективность для работы с реляционными структурами данных в распределенных сетях. Моделирование процессов в нотации BPMN обеспечило четкое понимание логики работы системы, что стало фундаментом для последующего проектирования.

В рамках практической части была разработана архитектура базы данных, включающая 28 таблиц. Приведение структуры к третьей нормальной форме позволило исключить избыточность данных и обеспечить высокую целостность информации. Особое внимание было уделено физической реализации: с помощью SQL-скриптов настроены связи, определены типы данных и установлены ограничения, исключающие появление логических ошибок.

Завершающим этапом стало тестирование системы. Проверка работоспособности базы данных осуществлялась через выполнение контрольных SQL-запросов, обеспечивающих выборку объектов по сложным критериям и формирование аналитической отчетности. Результаты тестирования подтвердили корректность спроектированной модели и её готовность к интеграции с серверной логикой приложения. Таким образом, разработанный продукт является законченным проектным решением, оптимизирующим процессы учета и бронирования жилой недвижимости.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бьюли, А. Изучаем SQL. Генерация, выборка и обработка данных / А. Бьюли. – Санкт-Петербург: Питер, 2021. – 384 с.
2. Гагарина, Л. Г. Технология разработки программного обеспечения / Л. Г. Гагарина. – Москва: ФОРУМ: ИНФРА-М, 2023. – 400 с.
3. Документация пользователя DBeaver Community [Электронный ресурс]. – Режим доступа: <https://dbeaver.com/docs/wiki/> (дата обращения: 29.03.2026).
4. Кузнецов, С. Д. Основы баз данных: учебное пособие / С. Д. Кузнецов. – Москва: ИНТУИТ, 2021. – 284 с.
5. Луис, Д. PostgreSQL: архитектура и реализация / Д. Луис. – Москва: ДМК Пресс, 2022. – 412 с.
6. Моргунов, Е. П. PostgreSQL. Основы языка SQL: учебное пособие / Е. П. Моргунов. – Санкт-Петербург: БХВ-Петербург, 2023. – 336 с.
7. Официальная документация PostgreSQL [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/> (дата обращения: 30.03.2026).
8. Спецификация нотации BPMN (Business Process Model and Notation) [Электронный ресурс]. – Режим доступа: <https://www.omg.org/spec/BPMN/2.0/> (дата обращения: 02.04.2026).

ПРИЛОЖЕНИЕ

Приложение 1

Листинг SQL-кода для создания базы данных.

-- Таблица strana

```
CREATE TABLE strana (  
    id          SERIAL PRIMARY KEY,  
    nazvanie_strany VARCHAR(100) NOT NULL UNIQUE  
);
```

-- Таблица region

```
CREATE TABLE region (  
    id          SERIAL PRIMARY KEY,  
    strana_id   INTEGER NOT NULL,  
    nazvanie_regiona VARCHAR(200) NOT NULL,  
    UNIQUE (strana_id, nazvanie_regiona),  
    FOREIGN KEY (strana_id) REFERENCES strana(id) ON DELETE CASCADE  
);
```

-- Таблица gorod

```
CREATE TABLE gorod (  
    id          SERIAL PRIMARY KEY,  
    region_id   INTEGER NOT NULL,  
    nazvanie_goroda VARCHAR(200) NOT NULL,  
    UNIQUE (region_id, nazvanie_goroda),  
    FOREIGN KEY (region_id) REFERENCES region(id) ON DELETE CASCADE  
);
```

-- Таблица gorodstrana

```
CREATE TABLE gorodstrana (  
    gorod_id   INTEGER NOT NULL,  
    strana_id  INTEGER NOT NULL,  
    PRIMARY KEY (gorod_id, strana_id),
```

```

FOREIGN KEY (gorod_id) REFERENCES gorod(id) ON DELETE CASCADE,
FOREIGN KEY (strana_id) REFERENCES strana(id) ON DELETE CASCADE
);
-- Таблица tip_rol
CREATE TABLE tip_rol (
    id          SERIAL PRIMARY KEY,
    nazvanie_rol VARCHAR(50) NOT NULL UNIQUE,
    opisaniye   TEXT
);
-- Таблица object_bd
CREATE TABLE object_bd (
    id          SERIAL PRIMARY KEY,
    nazvanie_tablitsy VARCHAR(50) NOT NULL,
    nazvanie_atributa VARCHAR(50) NOT NULL,
    UNIQUE (nazvanie_tablitsy, nazvanie_atributa)
);
-- Таблица polzovatel
CREATE TABLE polzovatel (
    id          SERIAL PRIMARY KEY,
    login       VARCHAR(20) NOT NULL UNIQUE,
    parol       VARCHAR(100) NOT NULL,
    email       email_address NOT NULL UNIQUE,
    telefon     phone_number NOT NULL UNIQUE,
    imya        VARCHAR(30) NOT NULL,
    familiya    VARCHAR(30) NOT NULL,
    data_rozhdeniya DATE,
    photo       VARCHAR(500),
    created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

```

-- Таблица rol_polzovatelya

```
CREATE TABLE rol_polzovatelya (  
    id          SERIAL PRIMARY KEY,  
    polzovatel_id  INTEGER NOT NULL,  
    tip_rol_id    INTEGER NOT NULL,  
    data_naznacheniya  TIMESTAMP      NOT      NULL      DEFAULT  
CURRENT_TIMESTAMP,  
    data_snyatiya  TIMESTAMP,  
    CHECK (data_snyatiya IS NULL OR data_snyatiya > data_naznacheniya),  
    FOREIGN KEY (polzovatel_id) REFERENCES polzovatel(id) ON DELETE  
CASCADE,  
    FOREIGN KEY (tip_rol_id) REFERENCES tip_rol(id) ON DELETE  
CASCADE  
);
```

-- Таблица administrator

```
CREATE TABLE administrator (  
    polzovatel_id  INTEGER PRIMARY KEY,  
    posledniy_vhod  TIMESTAMP NOT NULL,  
    FOREIGN KEY (polzovatel_id) REFERENCES polzovatel(id) ON DELETE  
CASCADE  
);
```

-- Таблица moderator

```
CREATE TABLE moderator (  
    polzovatel_id  INTEGER PRIMARY KEY,  
    data_naznacheniya  DATE NOT NULL DEFAULT CURRENT_DATE,  
    posledniy_vhod  TIMESTAMP,  
    kolichestvo_proverok  INTEGER NOT NULL DEFAULT 0 CHECK  
(kolichestvo_proverok >= 0),
```



```

    FOREIGN KEY (polzovatel_id) REFERENCES polzovatel(id) ON DELETE
CASCADE
);
-- Таблицаarendodatel
CREATE TABLEarendodatel (
    polzovatel_id      INTEGER PRIMARY KEY,
    otchestvo          VARCHAR(30),
    kolichestvo_obyavleniy  INTEGER NOT NULL DEFAULT 0 CHECK
(kolichestvo_obyavleniy >= 0),
    FOREIGN KEY (polzovatel_id) REFERENCES polzovatel(id) ON DELETE
CASCADE
);
-- Таблицаarendator
CREATE TABLEarendator (
    polzovatel_id INTEGER PRIMARY KEY,
    otchestvo     VARCHAR(30) NOT NULL,
    FOREIGN KEY (polzovatel_id) REFERENCES polzovatel(id) ON DELETE
CASCADE
);
-- Таблицаnalichiye_zhivotnykh
CREATE TABLEnalichiye_zhivotnykh (
    id      SERIAL PRIMARY KEY,
    nazvanie VARCHAR(50) NOT NULL UNIQUE
);
-- Таблицаnalichiye_zhivotnykharendator
CREATE TABLEnalichiye_zhivotnykharendator (
    nalichiye_zhivotnykh_id INTEGER NOT NULL,
    arendator_polzovatel_id INTEGER NOT NULL,
    PRIMARY KEY (nalichiye_zhivotnykh_id, arendator_polzovatel_id),

```

```

FOREIGN KEY (nalichiye_zhivotnykh_id) REFERENCES
nalichiye_zhivotnykh(id) ON DELETE CASCADE,
FOREIGN KEY (arendator_polzovatel_id) REFERENCES
arendator(polzovatel_id) ON DELETE CASCADE
);
-- Таблица nalichiye_detey
CREATE TABLE nalichiye_detey (
    id SERIAL PRIMARY KEY,
    nalichiye VARCHAR(50) NOT NULL,
    opisanie TEXT
);
-- Таблица arendator_nalichiye_detey
CREATE TABLE arendator_nalichiye_detey (
    arendator_polzovatel_id INTEGER NOT NULL,
    nalichiye_detey_id INTEGER NOT NULL,
    PRIMARY KEY (arendator_polzovatel_id, nalichiye_detey_id),
    FOREIGN KEY (arendator_polzovatel_id) REFERENCES
arendator(polzovatel_id) ON DELETE CASCADE,
    FOREIGN KEY (nalichiye_detey_id) REFERENCES nalichiye_detey(id) ON
DELETE CASCADE
);
-- Таблица kartochka_kvartiry
CREATE TABLE kartochka_kvartiry (
    id SERIAL PRIMARY KEY,
    arendodatel_id INTEGER NOT NULL,
    gorod_id INTEGER NOT NULL,
    opisaniye_kvartiry TEXT,
    adres_kvartiry TEXT NOT NULL,
    ploshad INTEGER NOT NULL CHECK (ploshad > 0),

```

tsena DECIMAL(10,2) NOT NULL CHECK (tsena >= 0),
 komnat positive_integer NOT NULL,
 etazh positive_integer NOT NULL,
 etazhnost positive_integer NOT NULL,
 remont VARCHAR(50),
 mebel SMALLINT NOT NULL DEFAULT 0 CHECK (mebel IN (0,
 1)),
 tehnika SMALLINT NOT NULL DEFAULT 0 CHECK (tehnika IN (0,
 1)),
 internet SMALLINT NOT NULL DEFAULT 0 CHECK (internet IN (0,
 1)),
 parkovka SMALLINT NOT NULL DEFAULT 0 CHECK (parkovka IN
 (0, 1)),
 lift SMALLINT NOT NULL DEFAULT 0 CHECK (lift IN (0, 1)),
 balkon SMALLINT NOT NULL DEFAULT 0 CHECK (balkon IN (0,
 1)),
 zhivotnye_dopustimo SMALLINT NOT NULL DEFAULT 0 CHECK
 (zhivotnye_dopustimo IN (0, 1)),
 deti_dopustimy SMALLINT NOT NULL DEFAULT 1 CHECK
 (deti_dopustimy IN (0, 1)),
 kuriye_dopustimo SMALLINT NOT NULL DEFAULT 0 CHECK
 (kuriye_dopustimo IN (0, 1)),
 min_srok_v_sutkah positive_integer NOT NULL DEFAULT 1,
 zalog DECIMAL(10,2),
 CHECK (etazh <= etazhnost),
 UNIQUE (arendodatel_id, adres_kvartiry),
 FOREIGN KEY (arendodatel_id) REFERENCES arendodatel(polzovatel_id) ON
 DELETE CASCADE,
 FOREIGN KEY (gorod_id) REFERENCES gorod(id) ON DELETE RESTRICT

```

);
-- Таблица photo_kvartiry
CREATE TABLE photo_kvartiry (
    id          SERIAL PRIMARY KEY,
    kartochka_kvartiry_id  INTEGER NOT NULL,
    url_photo   VARCHAR(500) NOT NULL,
    poryadok    INTEGER NOT NULL DEFAULT 0,
    FOREIGN KEY (kartochka_kvartiry_id) REFERENCES kartochka_kvartiry(id)
ON DELETE CASCADE
);
-- Таблица status_bronirovaniya
CREATE TABLE status_bronirovaniya (
    id    INTEGER PRIMARY KEY,
    nazvanie VARCHAR(50) NOT NULL
);
-- Таблица bronirovaniye
CREATE TABLE bronirovaniye (
    id          SERIAL PRIMARY KEY,
    arendator_polzovatel_id  INTEGER NOT NULL,
    kartochka_kvartiry_id  INTEGER NOT NULL,
    status_bronirovaniya_id  INTEGER NOT NULL,
    data_zayezda    DATE NOT NULL,
    data_vyyezda    DATE NOT NULL,
    created_at      TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP,
    CHECK (data_vyyezda > data_zayezda),
    UNIQUE (kartochka_kvartiry_id, data_zayezda, data_vyyezda),
    FOREIGN KEY      (arendator_polzovatel_id)      REFERENCES
arendator(polzovatel_id) ON DELETE CASCADE,

```

```

    FOREIGN KEY (kartochka_kvartiry_id) REFERENCES kartochka_kvartiry(id)
ON DELETE CASCADE,
    FOREIGN KEY (status_bronirovaniya_id) REFERENCES
status_bronirovaniya(id)
);
-- Таблица status_predoplaty
CREATE TABLE status_predoplaty (
    id SERIAL PRIMARY KEY,
    nazvanie_statusa VARCHAR(50) NOT NULL UNIQUE,
    opisanie TEXT
);
-- Таблица sposob_oplaty
CREATE TABLE sposob_oplaty (
    id SERIAL PRIMARY KEY,
    nazvanie VARCHAR(50) NOT NULL UNIQUE
);
-- Таблица predoplata
CREATE TABLE predoplata (
    id SERIAL PRIMARY KEY,
    bronirovaniye_id INTEGER NOT NULL UNIQUE,
    status_predoplaty_id INTEGER NOT NULL,
    sposob_oplaty_id INTEGER,
    summa DECIMAL(10,2) NOT NULL CHECK (summa >= 0),
    data_platezha TIMESTAMP,
    nomer_transaktsii VARCHAR(50) UNIQUE,
    FOREIGN KEY (bronirovaniye_id) REFERENCES bronirovaniye(id) ON
DELETE CASCADE,
    FOREIGN KEY (status_predoplaty_id) REFERENCES status_predoplaty(id) ON
DELETE RESTRICT,

```

```

    FOREIGN KEY (sposob_oplaty_id) REFERENCES sposob_oplaty(id) ON
DELETE RESTRICT
);
-- Таблица reyting_klassifikator
CREATE TABLE reyting_klassifikator (
    id SERIAL PRIMARY KEY,
    otsenka INTEGER NOT NULL UNIQUE CHECK (otsenka BETWEEN 1 AND
5),
    opisanie VARCHAR(100)
);
-- Таблица otzivy
CREATE TABLE otzivy (
    id SERIAL PRIMARY KEY,
    moderator_polzovatel_id INTEGER,
    bronirovaniye_id INTEGER NOT NULL UNIQUE,
    reyting_klassifikator_id INTEGER NOT NULL,
    tekst_otzyva TEXT NOT NULL,
    date TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP,
    odobreno BIT NOT NULL DEFAULT B'0',
    FOREIGN KEY (moderator_polzovatel_id) REFERENCES
moderator(polzovatel_id) ON DELETE SET NULL,
    FOREIGN KEY (bronirovaniye_id) REFERENCES bronirovaniye(id) ON
DELETE CASCADE,
    FOREIGN KEY (reyting_klassifikator_id) REFERENCES
reyting_klassifikator(id) ON DELETE RESTRICT
);
-- Таблица sluzhba_podderzhki
CREATE TABLE sluzhba_podderzhki (

```

```

id    SERIAL PRIMARY KEY,
login VARCHAR(50) NOT NULL UNIQUE,
telefon phone_number NOT NULL UNIQUE
);
-- Таблица zaprosi_podderzhki
CREATE TABLE zaprosi_podderzhki (
    id          SERIAL PRIMARY KEY,
    sluzhba_podderzhki_id INTEGER,
    bronirovaniye_id    INTEGER,
    kartochka_kvartiry_id INTEGER,
    polzovatel_id      INTEGER NOT NULL,
    opisanie_problemy   TEXT NOT NULL,
    data_podachi        DATE NOT NULL DEFAULT CURRENT_DATE,
    data_resheniya      DATE,
    kommentarii        TEXT,
    CHECK (data_resheniya IS NULL OR data_resheniya >= data_podachi),
    FOREIGN KEY (sluzhba_podderzhki_id) REFERENCES sluzhba_podderzhki(id)
ON DELETE SET NULL,
    FOREIGN KEY (bronirovaniye_id) REFERENCES bronirovaniye(id) ON
DELETE SET NULL,
    FOREIGN KEY (kartochka_kvartiry_id) REFERENCES kartochka_kvartiry(id)
ON DELETE SET NULL,
    FOREIGN KEY (polzovatel_id) REFERENCES polzovatel(id) ON DELETE
CASCADE
);
Заполнение таблиц базы данных.
-- Таблица strana
INSERT INTO strana (id, nazvanie_strany) VALUES
(1, 'Россия'),

```

(2, 'Беларусь'),
(3, 'Казахстан'),
(4, 'Германия'),
(5, 'Франция');

-- Таблица region

INSERT INTO region (id, strana_id, nazvanie_regiona) VALUES

(1, 1, 'Москва и Московская область'),
(2, 1, 'Санкт-Петербург и Ленинградская область'),
(3, 1, 'Республика Татарстан'),
(4, 1, 'Краснодарский край'),
(5, 2, 'Минская область'),
(6, 2, 'Гомельская область'),
(7, 3, 'Алматинская область'),
(8, 4, 'Бавария'),
(9, 4, 'Берлин'),
(10, 5, 'Иль-де-Франс');

-- Таблица город

INSERT INTO город (id, region_id, nazvanie_goroda) VALUES

(1, 1, 'Москва'),
(2, 1, 'Мытищи'),
(3, 2, 'Санкт-Петербург'),
(4, 3, 'Казань'),
(5, 4, 'Сочи'),
(6, 5, 'Минск'),
(7, 6, 'Гомель'),
(8, 7, 'Алматы'),
(9, 8, 'Мюнхен'),
(10, 9, 'Берлин');

-- Таблица городстрана


```

INSERT INTO gorodstrana (gorod_id, strana_id) VALUES
    (1, 1), (2, 1), (3, 1), (4, 1), (5, 1),
    (6, 2), (7, 2), (8, 3), (9, 4), (10, 4);
-- Таблица tip_rol
INSERT INTO tip_rol (id, nazvanie_rol, opisaniye) VALUES
    (1, 'Администратор', 'Полный доступ ко всем функциям системы'),
    (2, 'Модератор', 'Проверка объявлений и отзывов, решение споров'),
    (3, 'Арендодатель', 'Размещение объявлений о сдаче жилья'),
    (4, 'Арендатор', 'Поиск и бронирование жилья');
-- Таблица object_bd
INSERT INTO object_bd (id, nazvanie_tablitsy, nazvanie_atributa) VALUES
    (1, 'polzovatel', 'all'),
    (2, 'kartochka_kvartiry', 'all'),
    (3, 'kartochka_kvartiry', 'tsena'),
    (4, 'bronirovaniye', 'all'),
    (5, 'otzivy', 'all'),
    (6, 'zaprosi_podderzhki', 'all'),
    (7, 'polzovatel', 'login'),
    (8, 'polzovatel', 'email'),
    (9, 'kartochka_kvartiry', 'adres_kvartiry'),
    (10, 'kartochka_kvartiry', 'opisaniye_kvartiry');
-- Таблица polzovatel
INSERT INTO polzovatel (id, login, parol, email, telefon, imya, familiya,
data_rozhdeniya, photo, created_at) VALUES
    (1, 'admin', 'hash123', 'admin@mail.ru', '+70001112233', 'Иван', 'Иванов', '1985-
05-15', NULL, '2024-01-01 10:00:00'),
    (2, 'moder1', 'hash124', 'moder1@mail.ru', '+70001112234', 'Петр', 'Петров',
'1990-03-20', NULL, '2024-01-02 10:00:00'),

```

```

(3, 'owner1', 'hash125', 'owner1@yandex.ru', '+70001112235', 'Сергей', 'Сергеев',
'1982-11-10', '/uploads/avatars/owner1.jpg', '2024-01-03 10:00:00'),
(4, 'owner2', 'hash126', 'owner2@yandex.ru', '+70001112236', 'Анна', 'Антонова',
'1988-07-25', NULL, '2024-01-04 10:00:00'),
(5, 'tenant1', 'hash127', 'tenant1@mail.ru', '+70001112237', 'Алексей', 'Алексеев',
'1995-01-30', '/uploads/avatars/tenant1.jpg', '2024-01-05 10:00:00'),
(6, 'tenant2', 'hash128', 'tenant2@mail.ru', '+70001112238', 'Мария', 'Морозова',
'1992-09-12', NULL, '2024-01-06 10:00:00'),
(7, 'tenant3', 'hash129', 'tenant3@gmail.com', '+70001112239', 'Дмитрий',
'Дмитриев', '1998-04-18', NULL, '2024-01-07 10:00:00'),
(8, 'owner3', 'hash130', 'owner3@mail.ru', '+70001112240', 'Ольга', 'Орлова',
'1985-12-05', '/uploads/avatars/owner3.jpg', '2024-01-08 10:00:00'),
(9, 'moder2', 'hash131', 'moder2@platforma.ru', '+70001112241', 'Наталья',
'Наумова', '1991-06-22', NULL, '2024-01-09 10:00:00'),
(10, 'tenant4', 'hash132', 'tenant4@yandex.ru', '+70001112242', 'Павел', 'Павлов',
'1996-02-14', NULL, '2024-01-10 10:00:00');

```

-- Таблица rol_polzovatelya

```

INSERT INTO rol_polzovatelya (id, polzovatel_id, tip_rol_i_id, data_naznacheniya,
data_snyatiya) VALUES

```

```

(1, 1, 1, '2024-01-01 00:00:00', NULL),
(2, 2, 2, '2024-01-02 00:00:00', NULL),
(3, 3, 3, '2024-01-03 00:00:00', NULL),
(4, 4, 3, '2024-01-04 00:00:00', NULL),
(5, 5, 4, '2024-01-05 00:00:00', NULL),
(6, 6, 4, '2024-01-06 00:00:00', NULL),
(7, 7, 4, '2024-01-07 00:00:00', NULL),
(8, 8, 3, '2024-01-08 00:00:00', NULL),
(9, 9, 2, '2024-01-09 00:00:00', NULL),
(10, 10, 4, '2024-01-10 00:00:00', NULL);

```

-- Таблица administrator

```
INSERT INTO administrator (polzovatel_id, posledniy_vhod) VALUES  
    (1, '2026-03-10 10:00:00');
```

-- Таблица moderator

```
INSERT INTO moderator (polzovatel_id, data_naznacheniya, posledniy_vhod,  
kolichество_proverok) VALUES  
    (2, '2024-01-02', '2026-03-09 15:00:00', 150),  
    (9, '2024-01-09', '2026-03-08 12:00:00', 45);
```

-- Таблица arendodatel

```
INSERT INTO arendodatel (polzovatel_id, otchestvo, kolichество_obyavleniy)  
VALUES  
    (3, 'Иванович', 5),  
    (4, 'Сергеевна', 3),  
    (8, 'Петровна', 2);
```

-- Таблица arendator

```
INSERT INTO arendator (polzovatel_id, otchestvo) VALUES  
    (5, 'Алексеевич'),  
    (6, 'Дмитриевна'),  
    (7, 'Андреевич'),  
    (10, 'Павлович');
```

-- Таблица nalichiye_zhivotnykh

```
INSERT INTO nalichiye_zhivotnykh (id, nazvanie) VALUES  
    (1, 'Кот'),  
    (2, 'Собака'),  
    (3, 'Другое животное'),  
    (4, 'Нет животных');
```

-- Таблица nalichiye_zhivotnykharendator

```
INSERT INTO nalichiye_zhivotnykharendator (nalichiye_zhivotnykh_id,  
arendator_polzovatel_id) VALUES
```

```

(1, 5),
(4, 6),
(2, 7),
(3, 10),
(4, 5);
-- Таблица nalichiye_detey
INSERT INTO nalichiye_detey (id, nalichiye, opisanie) VALUES
(1, 'Нет детей', NULL),
(2, 'Есть дети', 'Один ребёнок 4 года'),
(3, 'Есть дети', 'Двое детей 6 и 9 лет'),
(4, 'Есть дети', 'Дети 7–14 лет'),
(5, 'Есть дети', 'Подростки старше 14 лет');
-- Таблица arendator_nalichiye_detey
INSERT INTO arendator_nalichiye_detey (arendator_polzovatel_id,
nalichiye_detey_id) VALUES
(5, 2),
(6, 1),
(7, 3),
(10, 4);
-- Таблица kartochka_kvartiry
INSERT INTO kartochka_kvartiry (
id, arendodatel_id, gorod_id, opisaniye_kvartiry, adres_kvartiry,
ploshad, tsena, komnat, etazh, etazhnost, remont,
mebel, tehnika, internet, parkovka, lift, balkon,
zhivotnye_dopustimo, deti_dopustimy, kuriye_dopustimo,
min_srok_v_sutkah, zalog
) VALUES

```

(1, 3, 1, 'Светлая двушка в центре Москвы, рядом с метро. Вся необходимая мебель и техника.', 'ул. Профсоюзная, 12', 52, 3800, 2, 6, 9, 'Евро', 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 5000),

(2, 3, 1, 'Студия в новом ЖК с современным ремонтом. Развитая инфраструктура.', 'Новоданиловская наб., 6', 38, 4500, 1, 12, 16, 'Дизайнерский', 1, 1, 1, 1, 1, 0, 0, 1, 0, 2, 10000),

(3, 4, 3, 'Трёшка у метро в Санкт-Петербурге. Просторная квартира.', 'Невский пр., 85', 68, 5200, 3, 4, 5, 'Косметический', 1, 1, 1, 0, 0, 1, 1, 1, 1, 3, 7000),

(4, 4, 4, 'Двушка в центре Казани, пешая доступность до Кремля.', 'ул. Баумана, 15', 55, 3500, 2, 5, 10, 'Евро', 1, 1, 1, 1, 1, 1, 1, 0, 1, 30, 4000),

(5, 8, 5, 'Апартаменты у моря в Сочи с видом на Чёрное море.', 'ул. Морская, 5', 45, 6000, 1, 7, 14, 'Современный', 1, 1, 1, 1, 0, 1, 1, 1, 0, 3, 8000),

(6, 3, 3, 'Уютная однушка в спальном районе Санкт-Петербурга.', 'Московский пр., 22', 33, 3200, 1, 3, 5, 'Косметический', 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 3000),

(7, 4, 4, 'Квартира с видом на Казанский Кремль. Высокий этаж.', 'ул. Кремлёвская, 10', 62, 4900, 2, 8, 12, 'Евро', 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 6000),

(8, 8, 5, 'Дом в пригороде Сочи с собственным участком и парковкой.', 'п. Горный, 7', 80, 7500, 4, 2, 2, 'Частный дом', 1, 1, 1, 1, 0, 1, 1, 1, 1, 5, 12000),

(9, 3, 6, 'Квартира в центре Минска на проспекте Независимости.', 'пр. Независимости, 30', 50, 4000, 2, 5, 9, 'Евро', 1, 1, 1, 0, 1, 1, 1, 1, 0, 2, 5000),

(10, 4, 7, 'Уютная двушка в центре Гомеля, недалеко от парка.', 'ул. Советская, 12', 48, 2800, 2, 4, 5, 'Косметический', 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 4000);

-- Таблица photo_kvartiry

INSERT INTO photo_kvartiry (id, kartochka_kvartiry_id, url_photo, poryadok)
VALUES

(1, 1, '/uploads/apartments/1/main.jpg', 0),

(2, 1, '/uploads/apartments/1/kitchen.jpg', 1),

(3, 1, '/uploads/apartments/1/bedroom.jpg', 2),

```
(4, 2, '/uploads/apartments/2/main.jpg', 0),
(5, 2, '/uploads/apartments/2/bathroom.jpg', 1),
(6, 3, '/uploads/apartments/3/main.jpg', 0),
(7, 3, '/uploads/apartments/3/kitchen.jpg', 1),
(8, 3, '/uploads/apartments/3/living.jpg', 2),
(9, 4, '/uploads/apartments/4/main.jpg', 0),
(10, 5, '/uploads/apartments/5/main.jpg', 0),
(11, 6, '/uploads/apartments/6/main.jpg', 0),
(12, 7, '/uploads/apartments/7/main.jpg', 0),
(13, 8, '/uploads/apartments/8/main.jpg', 0),
(14, 9, '/uploads/apartments/9/main.jpg', 0),
(15, 10, '/uploads/apartments/10/main.jpg', 0);
```

-- Таблица status_bronirovaniya

```
INSERT INTO status_bronirovaniya (id, nazvanie) VALUES
```

```
(1, 'Ожидает подтверждения'),
(2, 'Подтверждено'),
(3, 'Отменено'),
(4, 'Завершено')
```

```
ON CONFLICT (id) DO NOTHING;
```

-- Таблица bronirovaniye

```
INSERT INTO bronirovaniye (id, arendator_polzovatel_id, kartochka_kvartiry_id,
status_bronirovaniya_id, data_zayezda, data_vyyezda, created_at) VALUES
```

```
(1, 5, 1, 2, '2026-03-10', '2026-03-17', '2026-02-01 10:00:00'),
(2, 6, 2, 1, '2026-04-05', '2026-04-12', '2026-02-15 14:30:00'),
(3, 7, 3, 2, '2026-05-20', '2026-05-27', '2026-03-01 09:00:00'),
(4, 5, 4, 2, '2026-06-01', '2026-06-08', '2026-03-10 11:00:00'),
(5, 10, 5, 2, '2026-06-10', '2026-06-20', '2026-03-15 16:00:00'),
(6, 6, 6, 1, '2026-07-01', '2026-07-07', '2026-04-01 10:00:00'),
(7, 7, 7, 2, '2026-07-15', '2026-07-22', '2026-04-10 12:00:00'),
```

```

(8, 5, 8, 2, '2026-08-01', '2026-08-10', '2026-05-01 09:30:00'),
(9, 10, 9, 2, '2026-08-12', '2026-08-19', '2026-05-15 14:00:00'),
(10, 6, 10, 1, '2026-09-01', '2026-09-08', '2026-06-01 11:00:00');
-- Таблица status_predoplaty
INSERT INTO status_predoplaty (id, nazvanie_statusa, opisanie) VALUES
(1, 'Ожидает оплаты', 'Платёж ещё не поступил'),
(2, 'Оплачен', 'Средства получены'),
(3, 'Подтверждён', 'Оплата проверена'),
(4, 'Возвращён', 'Средства возвращены арендатору'),
(5, 'Отменён', 'Бронирование отменено до оплаты');
-- Таблица sposob_oplaty
INSERT INTO sposob_oplaty (id, nazvanie) VALUES
(1, 'Банковская карта'),
(2, 'Банковский перевод'),
(3, 'Наличные');
-- Таблица predoplata
INSERT INTO predoplata (id, bronirovaniye_id, status_predoplaty_id,
sposob_oplaty_id, summa, data_platezha, nomer_transaktsii) VALUES
(1, 1, 3, 1, 3800, '2026-03-05 14:30:00', 'TX001'),
(2, 2, 1, 2, 0, NULL, NULL),
(3, 3, 3, 3, 5200, '2026-05-15 09:45:00', 'CASH001'),
(4, 4, 2, 1, 3500, '2026-05-28 11:00:00', 'TX002'),
(5, 5, 3, 1, 6000, '2026-06-01 12:00:00', 'TX003'),
(6, 6, 1, 2, 0, NULL, NULL),
(7, 7, 3, 3, 4900, '2026-07-10 10:00:00', 'CASH002'),
(8, 8, 3, 1, 7500, '2026-07-25 15:30:00', 'TX004'),
(9, 9, 2, 1, 4000, '2026-08-05 09:00:00', 'TX005'),
(10, 10, 1, 2, 0, NULL, NULL);
-- Таблица reyting_klassifikator

```

```
INSERT INTO reyting_klassifikator (id, otsenka, opisanie) VALUES
```

```
(1, 1, 'Ужасно — не рекомендую'),
```

```
(2, 2, 'Плохо — есть серьёзные недостатки'),
```

```
(3, 3, 'Нормально — соответствует ожиданиям'),
```

```
(4, 4, 'Хорошо — приятно удивлён'),
```

```
(5, 5, 'Отлично — превзошло ожидания');
```

```
-- Таблица otzivy
```

```
INSERT INTO otzivy (id, moderator_polzovatel_id, bronirovaniye_id,  
reyting_klassifikator_id, tekst_otzyva, date, odobreno) VALUES
```

```
(1, 2, 1, 5, 'Отличная квартира, всё чисто, есть всё необходимое. Хозяин  
приветливый.', '2026-03-18 11:20:00', B'1'),
```

```
(2, 2, 2, 4, 'Хорошая студия, удобное расположение. Немного шумновато из-  
за стройки рядом.', '2026-04-13 14:10:00', B'1'),
```

```
(3, 9, 3, 3, 'Нормальная квартира, но требует небольшого ремонта. За свою  
цену нормально.', '2026-05-28 09:30:00', B'1'),
```

```
(4, 2, 4, 5, 'Супер! Всё понравилось, обязательно вернёмся!', '2026-06-09  
12:00:00', B'1'),
```

```
(5, 9, 5, 4, 'Понравилось, хороший вид из окна, всё чисто.', '2026-06-21  
15:00:00', B'1'),
```

```
(6, 2, 6, 3, 'Так себе, мебель старая, нужен ремонт.', '2026-07-08 10:00:00',  
B'0'),
```

```
(7, 2, 7, 5, 'Шикарная квартира, отличный ремонт, всё новое.', '2026-07-23  
16:00:00', B'1'),
```

```
(8, 9, 8, 4, 'Хороший дом, тихий район, есть всё для комфортного  
проживания.', '2026-08-11 13:00:00', B'1'),
```

```
(9, 2, 9, 5, 'Всё отлично, чисто, уютно, рекомендую!', '2026-08-20 11:00:00',  
B'1'),
```

```
(10, 9, 10, 2, 'Не понравилось, грязный подъезд, сломанный кран.', '2026-09-09  
09:00:00', B'0');
```


-- Таблица sluzhba_podderzhki

INSERT INTO sluzhba_podderzhki (id, login, telefon) VALUES

(1, 'support_anna', '+70001112233'),

(2, 'support_ivan', '+70001112234'),

(3, 'support_maria', '+70001112235'),

(4, 'support_petr', '+70001112236'),

(5, 'support_elena', '+70001112237');

-- Таблица zaprosi_podderzhki

INSERT INTO zaprosi_podderzhki (

id, sluzhba_podderzhki_id, bronirovaniye_id, kartochka_kvartiry_id,

polzovatel_id, opisanie_problemy, data_podachi, data_resheniya, komentarii

) VALUES

(1, 1, 1, 1, 5, 'Не работает домофон, не могу попасть в подъезд', '2026-03-11', '2026-03-12', 'Связались с хозяином, проблема решена'),

(2, 2, 2, 2, 6, 'Вопрос по залогу: когда вернут после выезда?', '2026-04-06', NULL, 'Ожидаем ответа от арендодателя'),

(3, 3, 3, 3, 7, 'Сломался кран на кухне, течёт вода', '2026-05-21', '2026-05-23', 'Вызван сантехник, починили'),

(4, 1, 4, 4, 5, 'Шумные соседи сверху, не дают спать ночью', '2026-06-02', NULL, 'Направлено обращение в управляющую компанию'),

(5, 2, 5, 5, 10, 'Проблемы с интернетом, часто отключается', '2026-06-12', '2026-06-13', 'Провайдер провёл диагностику, восстановил соединение'),

(6, 3, 6, 6, 6, 'Нет горячей воды уже второй день', '2026-07-02', '2026-07-03', 'Авария на котельной, устранена'),

(7, 1, 7, 7, 7, 'Сломался кондиционер, в квартире жарко', '2026-07-16', NULL, 'Ждём мастера для ремонта'),

(8, 2, 8, 8, 5, 'Проблемы с замком входной двери, трудно открыть', '2026-08-02', '2026-08-03', 'Вызван слесарь, замок заменён'),

(9, 3, 9, 9, 10, 'Вопрос по оплате: почему списали больше указанной суммы?',
'2026-08-13', '2026-08-14', 'Разобрались, ошибка банка, возврат сделан'),
(10, 1, 10, 10, 6, 'Течёт кран в ванной, нужно починить', '2026-09-02', NULL,
'Вызван сантехник, ожидаем');

Словарь данных базы данных

Таблица: polzovatel

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор пользователя	id	serial	Первичный ключ	+
2	Логин	login	varchar(20)	Уникальное имя для входа	+
3	Пароль	parol	varchar(100)	Хэш пароля пользователя	+
4	Электронная почта	email	varchar(255)	Тип email_address	+
5	Номер телефона	telefon	varchar(20)	Тип phone_number	+
6	Имя	imya	varchar(30)	Имя пользователя	+
7	Фамилия	familiya	varchar(30)	Фамилия пользователя	+
8	Дата рождения	data_rozhdeniya	date	Дата рождения пользователя	-
9	Фотография	photo	varchar(500)	Путь к файлу изображения	-
10	Дата создания	created_at	timestamp	Значение по умолчанию CURRENT_TIMESTAMP	+

Таблица: kartochka_kvartiry

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор квартиры	id	serial	Первичный ключ	+
2	Идентификатор арендодателя	arendodatel_id	integer	Внешний ключ, ссылка на arendodatel(polzovatel_id)	+
3	Идентификатор города	gorod_id	integer	Внешний ключ, ссылка на gorod(id)	+
4	Описание квартиры	opisaniye_kvartiry	text	Подробная информация об объекте	-
5	Адрес	adres_kvartiry	text	Полный адрес объекта	+
6	Площадь	ploshad	integer	В кв. метрах, > 0	+
7	Цена	tsena	decimal(10,2)	Стоимость аренды, >= 0	+
8	Количество комнат	komnat	integer	Положительное число	+
9	Этаж	etazh	integer	Текущий этаж	+
10	Этажность	etazhnost	integer	Всего этажей в доме	+
11	Наличие животных	zhivotnye_dopustimo	smallint	0 – нет, 1 – да	+
12	Можно с детьми	deti_dopustimy	smallint	0 – нет, 1 – да	+
13	Залог	zalog	decimal(10,2)	Сумма обеспечительного платежа	-

Таблица: bronirovaniye

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор бронирования	id	serial	Первичный ключ	+
2	Идентификатор арендатора	arendator_polzovatel_id	integer	Внешний ключ, ссылка на arendator(polzovatel_id)	+

3	Идентификатор квартиры	kartochka_kvartiry_id	integer	Внешний ключ, ссылка на kartochka_kvartiry(id)	+
4	Статус бронирования	status_bronirovaniya_id	integer	Внешний ключ, ссылка на status_bronirovaniya(id)	+
5	Дата заезда	data_zayezda	date	Дата начала аренды	+
6	Дата выезда	data_vyyezda	date	Дата окончания аренды	+
7	Дата создания заявки	created_at	timestamp	Значение по умолчанию CURRENT_TIMESTAMP	+

Таблица: predoplata

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор платежа	id	serial	Первичный ключ	+
2	Идентификатор бронирования	bronirovaniye_id	integer	Внешний ключ, ссылка на bronirovaniye(id)	+
3	Статус предоплаты	status_predoplaty_id	integer	Внешний ключ, ссылка на status_predoplaty(id)	+
4	Способ оплаты	sposob_oplaty_id	integer	Внешний ключ, ссылка на sposob_oplaty(id)	-
5	Сумма	summa	decimal(10,2)	Размер внесенных средств	+
6	Номер транзакции	nomer_transaktsii	varchar(50)	Уникальный номер из платежной системы	-

Таблица: otzivy

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор отзыва	id	serial	Первичный ключ	+
2	Идентификатор модератора	moderator_polzovatel_id	integer	Ссылка на модератора, проверившего отзыв	-
3	Идентификатор бронирования	bronirovaniye_id	integer	Связь с конкретным фактом проживания	+
4	Оценка	reyting_klassifikator_id	integer	Ссылка на справочник оценок (1-5)	+

5	Текст отзыва	tekst_otzyva	text	Комментарий арендатора	+
6	Признак одобрения	odobreno	bit	В'0' — на проверке, В'1' — опубликован	+

Таблица: administrator

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор пользователя	polzovatel_id	integer	PK, FK -> polzovatel(id)	+
2	Время последнего входа	posledniy_vhod	timestamp	Фиксация активности	+

Таблица: moderator

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор пользователя	polzovatel_id	integer	PK, FK -> polzovatel(id)	+
2	Дата назначения	data_naznacheniya	date	По умолчанию CURRENT_DATE	+
3	Последний вход	posledniy_vhod	timestamp		-
4	Кол-во проверок	kolichество_proverok	integer	Счетчик модераций (>= 0)	+

Таблица: arendodatel

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор пользователя	polzovatel_id	integer	PK, FK -> polzovatel(id)	+
2	Отчество	otchestvo	varchar(30)		-
3	Количество объявлений	kolichество_obyavleniy	integer	Счетчик активных лотов (>= 0)	+

Таблица: arendator

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор пользователя	polzovatel_id	integer	ПК, FK -> polzovatel(id)	+
2	Отчество	otchestvo	varchar(30)		+

Таблица: nalichiye_zhivotnykharendator

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	ID типа животного	nalichiye_zhivotnykh_id	integer	Составной ПК, FK -> nalichiye_zhivotnykh(id)	+
2	ID арендатора	arendator_polzovatel_id	integer	Составной ПК, FK -> arendator(polzovatel_id)	+

Таблица: arendator_nalichiye_detey

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	ID арендатора	arendator_polzovatel_id	integer	Составной ПК, FK -> arendator(polzovatel_id)	+
2	ID статуса детей	nalichiye_detey_id	integer	Составной ПК, FK -> nalichiye_detey(id)	+

Таблица: sluzhba_podderzhki

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор сотрудника	id	serial	Первичный ключ	+
2	Логин	login	varchar(50)	Уникальное имя в системе	+
3	Телефон	telefon	phone_number	Рабочий номер	+

Таблицы-классификаторы (справочники)

Таблица: strana

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор страны	id	serial	Первичный ключ	+
2	Название страны	nazvanie_strany	varchar(100)	Уникальное наименование	+

Таблица: region

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор региона	id	serial	Первичный ключ	+
2	Внешний ключ на страну	strana_id	integer	Связь с таблицей strana	+
3	Название региона	nazvanie_regiona	varchar(200)	Область, край или республика	+

Таблица: gorod

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор города	id	serial	Первичный ключ	+
2	Внешний ключ на регион	region_id	integer	Связь с таблицей region	+
3	Название города	nazvanie_goroda	varchar(200)	Наименование населенного пункта	+

Таблица: tip_rolі

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор роли	id	serial	Первичный ключ	+
2	Название роли	nazvanie_rolі	varchar(50)	Админ, Модератор, Арендатор и др.	+
3	Описание полномочий	opisanie	text	Описание прав доступа	-

Таблица: status_bronirovaniya

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор статуса	id	integer	Первичный ключ	+
2	Название статуса	nazvanie	varchar(50)	Напр. "Подтверждено"	+

Таблица: status_predoplaty

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор статуса	id	serial	Первичный ключ	+
2	Название статуса	nazvanie_statusa	varchar(50)	Оплачен, возвращен и т.д.	+
3	Описание	opisanie	text	Пояснение смысла статуса	-

Таблица: sposob_oplaty

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор способа	id	serial	Первичный ключ	+
2	Название способа	nazvanie	varchar(50)	Карта, перевод, наличные	+

Таблица: object_bd

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор записи	id	serial	Первичный ключ	+
2	Название таблицы	nazvanie_tablitsy	varchar(50)	Имя сущности в БД	+
3	Название атрибута	nazvanie_atributa	varchar(50)	Имя конкретного поля или "all"	+

Таблица: reyting_klassifikator

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор записи	id	serial	Первичный ключ	+
2	Оценка цифрой	otsenka	integer	Число от 1 до 5	+
3	Описание оценки	opisanie	varchar(100)	Текстовая расшифровка	-

Таблица: nalichiye_zhivotnykh

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор записи	id	serial	Первичный ключ	+
2	Вид животного	nazvanie	varchar(50)	Кот, собака и т.д.	+

Таблица: nalichiye_detey

№	Описание	Имя поля	Тип	Примечание	Обяз.
1	Идентификатор записи	id	serial	Первичный ключ	+
2	Наличие детей (статус)	nalichiye	varchar(50)	Категория (нет детей, один и т.д.)	+
3	Описание	opisanie	text	Дополнительные детали	-

Все данные из классификаторов

Данные классификатора strana

id	nazvanie_strany
1	Россия
2	Беларусь
3	Казахстан
4	Германия
5	Франция

Данные классификатора region

id	strana_id	nazvanie_regiona
1	1	Москва и Московская область
2	1	Санкт-Петербург и Ленинградская область
3	1	Республика Татарстан
4	1	Краснодарский край
5	2	Минская область
6	2	Гомельская область
7	3	Алматинская область
8	4	Бавария
9	4	Берлин
10	5	Иль-де-Франс

Данные классификатора город

id	region_id	nazvanie_goroda
1	1	Москва
2	1	Мытищи
3	2	Санкт-Петербург
4	3	Казань
5	4	Сочи
6	5	Минск
7	6	Гомель
8	7	Алматы
9	8	Мюнхен

10	9	Берлин
----	---	--------

Данные классификатора tip_rolі

id	nazvanіe_rolі	opisanіe
1	Адміністратор	Полный доступ ко всем функциям системы
2	Модератор	Проверка объявлений и отзывов, решение споров
3	Арендодатель	Размещение объявлений о сдаче жилья
4	Арендатор	Поиск и бронирование жилья

Данные классификатора nalichiye_zhivotnykh

id	nazvanіe
1	Кот
2	Собака
3	Другое животное
4	Нет животных

Данные классификатора nalichiye_detey

id	nalichiye	opisanіe
1	Нет детей	NULL
2	Есть дети	Один ребёнок 4 года
3	Есть дети	Двое детей 6 и 9 лет
4	Есть дети	Дети 7–14 лет
5	Есть дети	Подростки старше 14 лет

Данные классификатора status_predoplaty

id	nazvanіe_statusa	opisanіe
1	Ожидает оплаты	Платёж ещё не поступил
2	Оплачен	Средства получены
3	Подтверждён	Оплата проверена
4	Возвращён	Средства возвращены арендатору
5	Отменён	Бронирование отменено до оплаты

Данные классификатора status_bronirovaniya

id	nazvanie	id
1	Подтверждено	1
2	В ожидании	2
3	Отменено	3
4	Завершено	4

Данные классификатора `sposob_oplaty`

id	nazvanie	id
1	Банковская карта	1
2	Банковский перевод	2
3	Наличные	3

Данные классификатора `reyting_klassifikator`

id	otsenka	opisanie
1	1	Ужасно — не рекомендую
2	2	Плохо — есть серьёзные недостатки
3	3	Нормально — соответствует ожиданиям
4	4	Хорошо — приятно удивлён
5	5	Отлично — превзошло ожидания

Данные классификатора `object_bd`

id	nazvanie_tablitsy	nazvanie_atributa
1	polzovatel	all
2	kartochka_kvartiry	all
3	kartochka_kvartiry	tsena
4	bronirovaniye	all
5	otzivy	all
6	zaprosi_podderzhki	all
7	polzovatel	login
8	polzovatel	email
9	kartochka_kvartiry	adres_kvartiry
10	kartochka_kvartiry	opisaniye_kvartiry